Cluster Computing (2025) 28:535
https://doi.org/10.1007/s10586-025-05207-1

Defenses against model stealing attacks in MLaaS: literature review and challenges

Aouatef Mahani[1] · Okba Kazar[2]

Received: 28 August 2024 / Revised: 7 January 2025 / Accepted: 14 February 2025 / Published online: 19 August 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
MLaaS (Machine Learning as a Service) platforms provide convenient and scalable options for deploying machine learning models. Nevertheless, these offerings are becoming more vulnerable to model theft attacks, in which enemies can withdraw exclusive models by requesting information from the service. This review evaluates the present status of protections against model theft attacks in MLaaS. At times, someone working with machine learning is either highly knowledgeable in the field or needs to utilize costly resources to manage the vast amount of data. Individuals utilize MLaaS (Machine Learning as a Service) to avoid these problems by accessing various services. We start by classifying the different ways attacks can happen, focusing on the methods adversaries use to create accurate models. Next, we will examine the defensive tactics suggested in the literature, such as detection mechanisms, response plans, and preventive actions. The effectiveness and feasibility of key methods such as limiting queries, altering query responses, and using strong watermarks are assessed. This review aims to offer a thorough comprehension of the threat landscape and provide practitioners with insights into the current best practices for securing machine learning models in a service-oriented setting. Exactly, this study focuses on a specific service - classification - where MLaaS offers customers a black-box model. The predictions are generated by the model in response to API queries. Regrettably, these forecasts are utilized by malicious customers to either gain access to private data or develop a replacement model. Model theft attacks, also known as model extraction attacks, pose a threat to the confidentiality of machine learning models and need to be safeguarded against. Different suggestions and countermeasures are classified in response to this threat.

Keywords Cloud computing · Cloud service · Classification · Surrogate model

1 Introduction

There are several products of MLaaS such as Microsoft Azure Machine Learning [1], and Google Prediction API [2]. They offer a set of services, such as data transformation, predictive analysis, and classification. As the use of ML requires knowledge in this field, and generally massive data may be processed, which require very powerful hardware resources and high costs, MLaaS is widely used and its demands are always growing. In this paper, we focus on the classification task in which MLaaS allows clients to use black-box models by sending queries via cloud-based prediction API only to get predictions. However, some malicious clients attack these models by using queries and their corresponding predictions to steal a model by building a surrogate model to avoid query payments, or to steal the privacy of the training dataset. This problem is known as model stealing attack or model extraction attack. This issue creates the need to protect the confidentiality of machine learning models, where several defenses are proposed, they are categorized in two big categories: real-time defenses and post stealing (reactive) ones. The first category includes passive, active, and pro-

Aouatef Mahani and Okba Kazar these authors contributed equally to this work.

✉ Aouatef Mahani
amahani@usthb.dz

Okba Kazar
okba.kazar@ukb.ac.ae

[1] Computer Sciences, University of Sciences and Technology HOUARI Boumediene, Bab Ezzouar, Algeries 16111, Algeria

[2] College of Arts, Sciences and Information Technology, University of Kalba, Sharjah, Sharjah, UAE

active defenses, and the second ones includes watermarking and dataset inference.

In the aim to present this problem and describe the different proposed methods and approaches, we organize this paper as follows: section 2 defines the stealing attack problem. Section 3 describes the different query strategies used by an attacker to steal a model. Section 4 provides taxonomy of different approaches and methods used to deal with the stealing attack problem, and it describes in detail each proposed one. Section 5 presents a synthetic study about the proposed methods and techniques. Section 6 contains experimental studies to show the different results obtained by different methods. Section 7 provides a conclusion and future researches.

2 Presentation of the model stealing attack problem

Nevertheless, the high computational expense makes it challenging for users to develop their own models using a training dataset with a large number of instances and extensive computational resources.Consequently, some models are deployed by some companies to Cloud Service Providers CSP in the aim to offer services by means of MLaaS [3]. These models are considered as blackbox, where a user sends one or several input queries to the model using cloud-based prediction API only to get predictions (outputs). However, some malicious clients use the set of inputs-outputs to label the substitute dataset in order to train a surrogate model, which is a stolen copy of the victim model [4]. As a result, this phenomenon leads to the appearance of a problem known as model theft attack or model extraction attacks [5, 6]. The objectives of an adversary are:

- Avoid the query charges.
- Violate the private training dataset.
- Use the stolen model to offer a competing service, which can be detrimental to business.
- Use the surrogate model to craft adversarial examples, or to create vulnerability for safety-critical applications such as malware detection.

3 The query strategies of the model extraction attacks

As it is explained in the previous section, a malicious client may use a several input queries from its own dataset to get predictions in the aim to build a surrogate model. According to the availability of the query dataset, the quality of queries and the method used to generate synthetic queries, several strategies exist such as surrogate base data-based schemes, Adversarial example-based schemes, and Synthetic data-based schemes. In the following subsections, we present each one.

3.1 Surrogate data-based schemes

In this strategy, the attacker may possess a dataset and he may select the suitable set of queries to get predictions from the victim model. Some attackers apply active learning strategies to enhance the selection of queries, such Knock-off [7] and ActiveThief [8].

3.1.1 Knock-off

Knock-off constructs a dataset knows as a transfer dataset in the aim to build a surrogate model with relevant queries. To this end, the image distribution $P_A(X)$ is selected at first, and then the transfer dataset is obtained by applying an adaptive strategy π to sample images. At each iteration t, the following steps are executed:

1. The policy module P_π produces a sample of images x_t to query the black box.
2. The image x_t is enriched by an associate label z_t to encourage relevant queries.
3. The reward signal r_t is shaped using multiple criteria.
4. The policy module is updated in the aim to maximize the expected reward.

3.1.2 ActiveThief

ActiveThief aims to build iteratively the set D using a thief dataset and an active learning strategy as follows:

1. Initially, the subset S_0 is obtained by a random selection from the initial thief dataset.
2. For the kth iteration (k = 0, 1, ..., N), these steps are executed:
   - The set D_k is constructed, such that D_k= (x, V(x), for x belongs to S_k) by querying the victim model V.
   - The substitute model F is trained on D_t (t=0, 1, ..., k).
   - The set R_k is constructed by querying the substitute model F on the set L_k, such that L_k contains samples that not belong to the set $S_0 \cup S_1 \cup ...S_k$.
   - The subset S_{k+1} is obtained from the set R_k using an active learning subset selection strategy, such Uncertainty strategy which is based on uncertainty sampling [9], and K-center strategy [10]

3.2 Adversarial example-based schemes

In this strategy, the adversary possesses a training dataset with small number of samples that represents the problem domain. As this dataset is insufficient, the adversary uses it to build a synthetic dataset, and it improves the adversarial samples in the aim to approximate the decision boundary of the target model. In the following two subsections, we present two methods of this strategy.

3.2.1 Jacobian-based augmentation JBA

JBA [11] applies data augmentation technique to iteratively expand a set of seed samples (known as seed query set) into a substitute training set. In each iteration of this technique, the attacker constructs at first new data samples near the model's decision boundary by augmenting existing samples. Then, he crafts the adversarial samples by implementing these two existing approaches: fast gradient sign method [12] and the algorithm of the Papernot [13].

3.2.2 FeatureFool FF

FF [14] is a method that enhances the efficiency of samples and reduces the number of queries when building the surrogate model. It adopts internal representation to generate a subset of synthetic samples, which will be used to efficiently learn the distance between decision boundaries of the victim model and the stolen model.

3.3 Synthetic data-based schemes

In this strategy, an adversary applies generative models to produce synthetic samples, even if he doesn't have any real data. In the following subsection, we present a method of this strategy.

Data-free substitute training DaST

DaST method [15] is used to generate adversarial examples to attack the victim model V, in which these examples have the same distribution as the samples used by the model V. To do, DaST generates the substitute model S, which will be used to generate adversarial examples to attack V by introducing the generative model G by executing these steps iteratively:

1. G randomly samples the noise vector z from the input space.
2. G produces the data X, such that X=G(z).
3. The adversary queries the model V to get the output: V(X).
4. The adversary trains the substitute model S using the pair (X, V(X)).
5. Update the substitute model S and the generative model G.

These steps are executed until the success rate of S doesn't increase or the iteration number is reached.

4 The proposed approaches and techniques

Given the significance of the issue, several strategies and tactics have been created in an effort to solve it. In general, the approaches and techniques can be divided into two categories; real-time defense and post-stealing defense. An approach is considered as a real-time defense if its objective consists to detect and prevent an attack when it is in progress. However, it is considered as a post-stealing defense if its objective is to prove the ownership of a model or of the private dataset in the event of stealing without protecting the model or preventing the theft. In the first category, there exist 03 kinds of defenses, which are passive defenses, active defenses, and pro-active defenses. The second category is known also as reactive defenses, which include DNN watermarking and dataset inference. Figure 1 provides a summary of each category's methodology. It should be noted that we employ acronyms that provide information about each procedure.

In the following subsections, we define each class of defense, and we present some methods and approach of each one.

4.1 Real-time defenses

4.1.1 Passive defenses

The methods of passive protection involve monitoring and analyzing the queries that are received with the intention of identifying and stopping hostile clients [6, 16–18]. We outline some of this category's methods and approaches below.

Extraction monitor:

Extraction Monitor (EM) [17] is a technique that uses a monitor to measure the model rate for the extracted model in the context of decision trees. To do this, it monitors queries with two different strategies. The first one uses all historical queries. However, the second one uses only a model summary of queries.For each strategy, EM monitors model extracted by both one user and colluding adversaries. In the following subsections, we detail each strategy.

Strategy 1: Model extraction warning using all historical queries

In this strategy, the monitor aims to detect the status of extracted model using the query set responses by

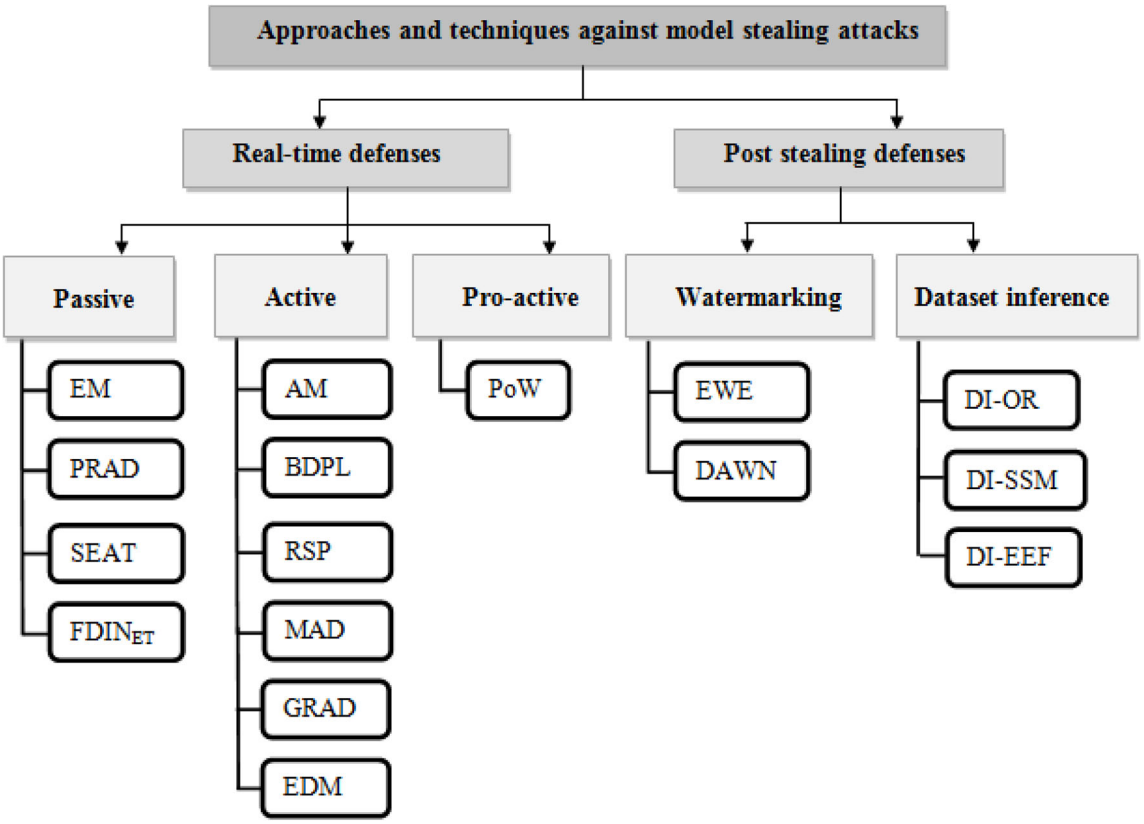


Fig. 1 Taxonomy of different defenses against model stealing attack

measuring the value of a new measure known as Extraction status.To do, EM follows these steps:

- The model owner possesses the validation set S that contains k classes and the decision tree T_O.
- The information gain of an attribute [19] is generalized to obtain a new measure known as Information Gain for a decision tree IG_{tree}, which is used to estimate the information gained by a user based on its queries.

For the tree T and the validation S, IG_{tree} is defined in the Equation 1:

$$IG_{tree}(S,T) = Entropy(S) - \sum_{l \in leaf(T)} \frac{|S_l|}{|S|} * Entropy(S_l) \quad (1)$$

Where: the validation set S is partitioned on several subsets S_l by the leaf nodes of T.

1. Model extraction warning for one user: The different steps of this process are as follows:

1. Construct the decision tree T_u for the user u using the query−response pairs.
2. Compute IG_{tree} for the tree T_u.
3. Compute the value of the Extraction Status measure for the user u, which is defined in the Equation 2:

$$Extraction_Status_u = \frac{IG_{tree}(S,T_u)}{IG_{tree}(S,T_0)} * 100 \quad (2)$$

4. If the value of the Extraction Status is more than the threshold t, than the monitor generates a model extraction warning, and it informs the model owners.

2. Model extraction status with collusion:

The monitor initially computes the extraction status for each potential combination with the goal of determining whether any k of the n users are conspiring to extract the model. The decision tree T_c is learnt on the set S_c for the subset C of k users. The sets of inquiries from each user in the subset C are concatenated to create this collection. Next, for every subset C, the IG_{tree} (S, S_c) is calculated. The extraction status is highly valuable to the collaborating enemies.

Strategy 2: Model extraction warning using compact user summaries

In this strategy, the monitor uses only the model summary of each user rather than the historical of all queries and their responses, knowing that a response contains the class label and the leaf id, which a query maps into decision tree T_0 of the model owner. The model summary of a user is represented by the matrix M of size lt*lf, where:

- lt: is the number of leaf nodes.
- lf: is the number of input features.
- M[i,j]: is an element of the matrix M that represents the linear constraint ([min,max]) on the feature j that the user has learned for leaf node i. This matrix is updated when a user sends a new query.

1. Model extraction warning for one user:

1. Construct the matrix M of the user u.
2. Compute the area of each leaf by aggregating leaves of the same class (see Equation 3)

$$Leaf_vol_i = \prod_{j \in f_i} abs(M(i,j) * max - M[i,j] * min), \forall i \in l_t \tag{3}$$

3. Compute the extraction percentage per class using the ratio of the area in the model summary to the area per class of the source model as shown in the Equation 4:

$$Ex_Per_Class_i = \frac{\sum_{l \in C_i} Leaf_Vol_i}{Class_Vol_{i]}} \tag{4}$$

4. Compute the extraction status of a user, which is a weighted sum of per-class extraction percentages by weighing each term with the class probability p_i as it is shown it Equation 5.
5. If extraction status is more than the threshold t, than the monitor generates a model extraction warning, and it informs the model owners.

$$Extraction_Status = \sum_{i \in C} p_i * Ext_Per_Class_i \tag{5}$$

2. Model extraction status with collusion

In this case, the same steps of strategy 1 are applied except that the maximum hyper volume is used instead of maximum Information Gain.

Protecting against DNN model stealing attacks PRADA

The method used to identify DNN model extraction assaults is PRADA [16]. To determine if a collection of API queries (S) deviates from the normal distribution, it entails analyzing S's distribution. To achieve this, the actions listed below are taken:

Step 1: construction of the set D that contains the minimum distances d_{min} as follows: For each sample x_i, that belongs to S, the minimum distance $d_{min}(x_i)$ between x_i and any previous sample x_0, x_1,, ...of the same class c is calculated. For efficiency, instead of using all past queries in S, the growing set G_c, is built for each class c. The set G_c contains only samples whose distance d_{min} is above a threshold value T_c. Initially, T_c is set to 0, but it is updated after adding each sample x_i to G_c. Also, $d_{min}(x_i)$ is added to the set D. The size of the set D may be higher than 100, because it is necessary to provide enough values to obtain a relevant value of the test statistic.

Step 2: construction of the set D' by removing the outliers from D. Outliers are values being more than 3 standard deviations away from the mean of values in D.

Step 3: Applying a normality test on the set D' to verify if it fits a normal distribution. The ShapiroWilk test [20] is used because the values of its test statistic W produced the largest difference when computed on benign and adversarial queries. W is defined on [0, 1] and a low value highlight deviation from a normal distribution. Finally, if W (D') is below a threshold δ, PRADA detects an extraction attack.

Similarity encoder by adversarial training SEAT:

The SEAT [18] is an algorithm that detects Jacobian-based model extraction attacks to defend against black-box models. It is composed of two components, which are similarity encoder and a detection scheme.

The role of the similarity encoder is the detection of similar pairs. To do, it compares the similarity of an incoming query to historical queries that belong to the same account. For the pair of query (q_i, q_j) that have the feature vectors $f(q_i)$ and $f(q_j)$, the l2 distance between $f(q_i)$ and $f(q_j)$ is computed. This pair is considered as similar if l2 distance is less than the threshold δ.

The role of detection scheme component is the monitoring of the number of similar pairs for each account. When this number exceeds some threshold of tolerance Nthresh, the detector raises an alarm, and the corresponding account is cancelled. The parameters Nthresh and δ are tuned by the defender to control the false positive rate (FPR).

FDINET: Protecting against DNN model extraction via feature distortion index:

FDINET [21] is a novel defense mechanism that leverages the feature distribution of deep neural network (DNN) models. It aims to detect if the queries come from benign or malicious clients, and to detect the colluding adversaries. To do this, FDINET includes three phases: (1) selection of anchor samples, (2) measuring feature distortion, (3) training extraction attack detector. In the following, we describe each phase.

Phase1: Selection of anchor samples

Since an adversary's objective is to get predictions and more information about a victim model, it may explore a large input space that contains samples. The feature distributions of these samples deviate from samples of original training dataset. Therefore, it is necessary to use a metric that measures the feature distributions between query samples and training dataset ones. To do, these following steps may be done:

- Select K samples from training dataset to construct the anchor set D_{anc}. These samples are known as anchor

samples, and they have a high-confidence to guarantee that they lie at the center of the considered set. Formally, a selected sample x is denoted as ($x_{c,j}$, c): j= 1...K, and c is the class of x.
– Use the selected anchor sample ($x_{c,j}$, c) to extract feature maps F_l^l ($x_{c,j}$) for layer l.

Phase 2: Measuring feature distortion

Feature distortion is defined as the deviation in the feature space between an anchor sample and a query sample. To measure this feature, a novel metric is proposed known as Feature Distribution Index "FDI", which is a vector of size K*L, where L is the number of layers; and each component concerns a sample j for the layer l. It is calculated by using the victim model F_v and the anchor set D_{anc}. For a sample j and the layer l, the component I_l^j is computed using the Equation 6:

$$I_j^l = d(\phi_v^l(x) - \phi_v^l(x_{c,j})); x_{c,j} \in D_{anc} \quad (6)$$

Where:

- ϕ_v^l: is the output feature F_v for layer l.
- d: the l2-norm in this paper.
- c: is the label of x predicted by F_v.

Phase 3: Training extraction attack detector

In this phase, the binary classifier C is built using the training dataset D_{train} as the negative class, the auxiliary dataset D_{aux} as the positive class, and the feature distortion index FDI. This classifier is used to detect extraction queries and to identify the colluding adversaries.

Since a query has low information entropy, it is a challenging to use a single query to detect if a client is malicious. To avoid this problem, a majority voting algorithm is applied on a batch of bs queries to compute the average confidence score AC of the predictions using the constructed binary classifier. Finally, a client is malicious if AC is higher than the threshold τ_1.

Identification of colluding adversaries

To detect if the clients u and v are colluding adversaries, the approach collects at first n*bs historical queries from each client to extract their n*bs FDI vectors I_u and I_v to apply two samples hypothesis test [22]. It consists to compute the values of the statistical test T and p-value at first, then to select the threshold τ_2 for statistical significance (usually τ_2=0.05) for testing. If p-value is below τ_2, then u and v are not colluding adversaries.

4.1.2 Active defenses

The general principle of active approaches is to perturb the output prediction. They return the perturbed output prediction rather returning the correct out prediction [23–25]. The approaches of this kind of defense can be also divided in general into two categories: accuracy-constrained defenses, and accuracy-preserving one.

Adaptive misinformation:

Adaptive Misinformation (AM) [23] is an accuracy-constrained defense, because it decreases the accuracy of the clone model by assigning an incorrect prediction y' to the queries that are out of distribution rather than a correct prediction y. Since, in perturbation based defenses, there is a signification amount of correlation between y and y', which leaks information about the original prediction that can be exploited by an adversary. To avoid this problem, AM produces the output y' which is uncorrelated with y. When AM receives a query x, it follows theses steps:

1. Determine if x is an Input Distribution ID or an Out Of Distribution OOD using the out of distribution detector.
2. If x is an ID input, then AM returns the correct prediction generated by the defender's model f.
3. If x is an OOD input, then AM returns the incorrect prediction generated by the combination of the model f and the misinformation function.

The objective of this detection is to assign a correct and confident prediction to an ID input, and a less confident prediction of an OOD input. Therefore, the detector may be able to distinguish between an ID input and OOD one. To achieve this objective, the defender's model is trained with Outlier Exposure [26].

In order to return an incorrect prediction for an OOD input and to make the training of the clone model very hard, the misinformation function is trained by minimizing the reverse cross entropy loss.

Boundary differentially private layer:

Attacker sends queries that are near to the decision boundary of a model, and it uses the corresponding responses to divulge the parameters of the own model. For this reason, Boundary Differentially Private Layer (BDPL) [25] detects at first the sensitive queries, then it obfuscates the output prediction of these queries using the notion of Boundary Differential Privacy BDP.

Reverse sigmoid perturbation:

Reverse Sigmoid Perturbations (RSP) [27] defense is an accuracy preserving defense that aims to maximize the loss of the stolen model, degrade and slow down the model stealing attacks, and to preserve accuracy. To this end, this method adds a noise to the output probabilities returned by neural networks to obtain the perturbed probabilities. These later should have the following properties: the sum across the dimensions must be 1, the magnitude of the perturbation may be controlled, and the accuracy should be preserved.

Maximum angular deviation between gradients:

Maximum Angular Deviation between gradients (MAD) [7] is a prediction poisoning approach, which is the first utilityconstrained defense in which the defender returns a perturbed prediction when it receives a query. It aims to achieve two objectives: utility and nonreplicability. The first objective concerns the returned prediction to serve benign users. Two utility measures are considered: accuracy of the defended model on test dataset, and the distance between the original (y) and perturbed (y') predictions may be controlled by adding perturbation magnitude (noted ϵ), such that $dist(y, y') = ||y - y'||p = \epsilon$. The second objective consists of to decrease the accuracy of the model's attacker FA on test dataset D_{test}.

Consequently, MAD considers model stealing defense as an Optimization Problem to poison the training data of an attacker by perturbing the predictions and maintaining the privacy and utility objectives. However, it is difficult to access to the training dataset of an attacker. Therefore, MAD simulates an attacker by another network, and it perturbs a prediction by adding targeted noise to the posterior probabilities to poison the adversary's gradient signal, which results in an adversarial gradient signal that maximally deviates from the original gradient to maximize the angular deviation between the original and the poisoned gradient signals.

Gradient redirection adversarial distillation defense:

The approach MAD computes the full Jacobian of the network's posterior. Consequently, its overhead cost is very high, and it depends on the number of labels. To overcome this drawback, a new approach is proposed, which is known as Gradient Redirection Adversarial Distillation Defense (GRAD) [28].

GRAD is a similar approach to MAD that redirects the gradient of an adversary in a targeted and efficient manner. To do, it applies an algorithm called gradient redirection algorithm to solve the problem of gradient redirection in an effective and optimal way, and it represents an adversary by a surrogate network, because the attacker's network is not accessible.

The utility objectives of GRAD are the classification error of defender on test dataset, and the distance between the original (y) and perturbed (y') prediction may be controlled by adding perturbation magnitude (noted ξ), such that $dist(y, y') = ||y - y'||p \leq \xi$.

Ensemble of diverse models:

The objective of Ensemble of Diverse Models (EDM) [24] is to decrease the accuracy of the clone model and to make model stealing hard to carry out. To this end, it trains several models to produce discontinuous output predictions.

During the training phase, EDM aims to achieve two objectives: accuracy objective, and diversity one. The first objective concerns examples that belong to the dataset D_{in} in the aim to obtain a high accuracy. However, the second one concerns examples that belong to the dataset D_{out} in the aim to produce dissimilar predictions.

To obtain discontinuous predictions, EDM selects one model based on the input query using a hashing function H to return the index of selected model.

The hashing function may be satisfying some properties such as security and transformation in variance to ensure that an adversary is unable to find it.

4.1.3 Pro-active defenses

Pro-active defense aims to prevent model stealing before it succeeds, without modifying the outputs, and without decreasing the accuracy of a model. In the next subsection, we present the first proactive defense, known as Proof-of-Work based defense.

Proof-of-work-based defense:

Proof-of-Work based defense (PoW-based defense) [29] is the first pro-active defense that aims to increase the computational cost of a model extraction without influencing the quality of the outputs. At this end, the proof-of-work(PoW) technique [30] is wrapped in the original model predictions to impose users to expand additional time for solving PoW puzzle. This defense takes into consideration the benign users where the expanding time depends of the cost of queries. This cost is computed by applying privacy metric via the PATE (Private Aggregation of Teacher Ensembles) framework [31].Consequently, a user takes more time to solve the puzzle if the privacy cost is higher. The steps of this defense are:

1. A user sends a set of queries to a machine learning service.
2. The privacy cost of these queries is computed and sent to the PoW server.
3. If the received queries are sent by an attacker, then the server adds a new cost to obtain the total query cost.
4. The PoW server generates a puzzle, whose difficulty is based on the total cost.
5. The user solves the puzzle using its PoW Client and sends the solution back to the server.
6. The PoW server checks the solution. If it is correct, then the victim model returns the predictions.

4.2 Post stealing (reactive) defenses:

Approaches and techniques of this kind of defenses are used only to prove the ownership of a model or of the private dataset in the event of stealing without protecting the model or preventing the theft. There are divided into

two classes: watermarking methods and dataset inferences ones.

4.2.1 Watermarking

Watermarks are considered as a type of markers, including videos, audios, or images, which are embedded in a signal or IP to demonstrate ownership of the object [32]. The set of the watermarks is known as Trigger set, and the function B that returns an incorrect prediction of each watermark x, where $B(x) \neq F_v(x)$, is called a backdoor function. Several methods and approaches are proposed that differ in the way of generation and embedding watermarks. In the following, we present some ones:

Entangled watermark embedding:

The watermarks generated by some watermarked methods are outliers compared to those of the inputs of training dataset. Consequently, an adversary ignores the watermarks and uses only the inputs from the task distribution when it builds its model. To avoid this problem, Entangled Watermark Embedding (EWE) [5] generates watermarks that have the same representation as task distribution by applying the Soft Nearest Neighbor Loss SNNL [33] to entangle representations extracted from training data and watermarks. Entanglement allows representing watermarks and training data similarly. Therefore, an adversary cannot distinguish between task distribution and watermarks, and it uses these later to build its model. To watermark a model, EWE follows these steps:

Step 1: Generation of watermarks

Watermarks are generated by selecting two similar classes that the model does not misclassify as follows:

1. Compute the average cosine similarity between each pair of classes, and select the pair that has a highest value, such that the samples that belong to these classes have a similar representation, and it is easy to entangle them. The selected pair is composed of target class C_t and source class C_s, it is noted (C_t, C_s).
2. Generate the set of watermarks by entangling samples of two selected classes, and insert watermarks between these classes.

Step 2: Modification of the loss function

This step aims to watermark the model robustly. At this end, SNNL is computed for each layer l using the layer's representation x_w^l of a batch of inputs x_w sampled from both the task distribution and set of watermarks, and a layer-specific temperature T^l. Then, the SNNL is summed across all layers l, and it is multiplied by a weight factor w to control how much importance is given to the SNNL relative to the cross-entropy during optimization.

To prove the ownership of the stolen model, two-sample T test [22] is applied using few queries and these two measures: watermark success rate, and false watermark rate. The first one represents the probability of correctly identifying watermarked (to class C_t), and the second one represents the probability of classifying data without the trigger in the target class C_t. If the watermark success rate is significantly higher than the false watermark rate, then the model is stolen.

Dynamic adversarial watermarking of neural network:

The Dynamic Adversarial Watermarking of Neural Network (DAWN) [32] is an approach that trains directly the model F_v without changes, and it doesn't generate watermarks in advance. That is because the watermarks are specific to each client by selecting dynamically a subset of queries from API clients and assigning to each selected query an incorrect prediction using the backdoor function.

To prove that the model F_s is surrogate of the victim model F_v, the trigger set is used to compute the ratio of different results between the backdoor function B(x) and the suspected surrogate model $F_s(x)$ for all inputs x in the trigger set. If this ratio is less than the value of the tolerated error rate then F_s is a surrogate of F_v. The value of e depends on the number of classes.

4.2.2 Datasets inference

In the context of supervised learning, dataset inference (DI) [34] is the verification-based defense against model theft. In the case of theft, its goal is to establish who owns the private dataset or victim model. This is accomplished by asserting that the surrogate model is a derivative of the own private training data utilizing a variety of strategies and tactics. We provide several techniques in the next subsections.

Dataset inference: Ownership resolution in machine learning DI-OR:

The objective of the method Dataset inference: Ownership Resolution in machine learning (DI-OR) [34] is to determine if the private dataset used to train the owner model f is also used by an adversary to train its model f_{A*}. To this end, the first key idea is that the objective of a classifier is to maximize the distance between the training samples and decision boundary, and an adversary will position this data far from this decision boundary. The second idea is that the data used to train the model f has a lower confidence score. Based on these ideas, this method compares the confidence scores obtained by querying a regression model using the samples that belong to the private dataset and a public dataset in the aim to prove the ownership of the private dataset. To do, the proposed method has two main steps: generation of embedding features and ownership tester.

Generation of embeddings: the feature embedding for the sample x and the model f is characterized by its prediction margin. To generate the feature embeddings, the two following methods are used: White-Box Setting: MinGD and Black-Box Setting: Blind Walk

Ownership tester: the ownership is tested in two phases: a regression model and hypothesis testing.

Phase 1: the regression model g_v is trained using embeddings and the ground truth membership labels, to predict a measure of confidence score that a sample contains private information.

Phase 2: two confidence score vectors c and c_v are constructed, to provide them to the two sample T-test (These vectors are obtained by querying models respectively from public and private datasets). Then, compute the mean confidence scores μ and μ_v that correspond to the vectors c and c_v respectively, and the p-value (For more details, see subsection 6.3.12).

Dataset inference for self-supervised models:

The Dataset Inference for Self-Supervised Models (DI-SSM) objective is to determine whether the model fv is used to construct the stolen model f_s. To this end, this method analyses at first the training and test datasets by computing their representations and modeling their densities, then it computes the value of the log-likelihood of an encoder's output representations [35].

The log-likelihood allows determining if whether the victim model is stolen. If the value of this measure of the representation on the victim's training dataset is higher than that on test dataset then the victim model is stolen.

Dataset inference using embedded external features: The method Dataset inference using Embedded External Features (DI-EEF) [36] aims to check the ownership of a victim model by embedding external features in that model. It includes 03 steps: (1) embedding of external features, (2) training a meta-classifier, and (3) verifying the ownership. In the following, we detail each step:

(1) Embedding external features: to embed the external features into the victim model V, we use the transformed data D_t and D_b to build V. The first dataset D_t is obtained as follows:

- Select randomly p% samples from the benign training data D to obtain the dataset D_s, where p is known as dubbed transformation rate.
- Use the style image x_s.
- Apply the style transformer T: $X \times X \rightarrow X$ on D_s to obtain D_t, such that: $D_t = (x\cdot, y)$ where $x\cdot = T(x, x_s)$ for each pair (x, y) that belongs to D_s.
- While, the second dataset is a subset of the original dataset, such that $D_b = D \setminus D_s$.

(2) Training a meta-classifier: Since the embedded external features can slightly influence the prediction, and these features cannot be presented by an expression, it is necessary to train an additional binary meta-classifier C_w, which trained on the dataset D_c. Where D_c is obtained by using the benign model B and the suspicious model V as follows (Equation 7):

$$D_c = (g_V(x'), +1) \bigsqcup (g_B(x'), +1), \text{ where} (x', y) \in D_t \tag{7}$$

Where:

$$g_V(x') = sgn(\bigtriangledown \Theta L(V(x'), y)), \tag{8}$$

$$g_B(x') = sgn(\bigtriangledown \Theta L(B(x'), y)) \tag{9}$$

Where: sgn(•) is sign function [37].

(3) Verifying the ownership: to determine if the suspicious model S is stolen from the victim model V, the transformed image x' and its label y is used to compute $C(g_s\ (x'))$, where $g_s\ (x')$is the sign obtained by the suspicious model. If $C(g_s\ (x')) = 1$ then S is stolen from V, otherwise it is not stolen.

In practice, the verification of the ownership is done by randomly sample m different transformed images from D_t to conduct the pair-wise [22] and calculate its p-value (for more details, see subsection 6.3.12)

5 Synthesis study

The proposed approaches and tactics used to deal with the problem of model stealing attacks are divided in four categories: passive, active, reactive, and pro-active defenses, and they differ on:

- The time of intervention: during an attack or after.
- The objectives: decreasing the accuracy of the stolen model, taking into consideration benign clients, monitoring and analysis the received queries to detect malicious clients, proving the ownership of the victim model and the private dataset, increasing the time of building a model.
- The reactions: perturbing the output predictions, canceling accounts, producing discontinuous output predictions, raising an alert.

6 Experimental studies

In order to present the results obtained by state of the art approaches and techniques, we have divided the work in 4 subsections. Subsections 6.1 describes the used datasets. Subsection 6.2 contains all attacks and models used by state of the art tactics. Subsection 6.3 contains several

evaluation measures used to evaluate the capacity or the success of an approach to protect a model. Subsection 6.4 presents the results obtained by the different methods.

6.1 Used datasets

In these studies, we have collected all datasets used by state of the art approaches and methods. Table 1 summarizes these datasets and shows their acronyms used in this section, the number of samples (#samples), and the number of classes (# classes)

6.2 Used attacks and models

Table 2 represents the different attacks and models used by state of the art approaches in the aim to prove their efficiency.

6.3 Evaluation metrics

It is necessary to use metrics to evaluate an approach to prove its robustness and its efficiency. In this subsection, we present all metrics used by all state of the art approaches.

6.3.1 1-Rtest

The measure 1-R_test [6] is used to compare the performance of the surrogate model S using the victim model V. It represents the accuracy of the surrogate model on test dataset T, which is computed as shown in the Equation 10.

$$R_{test} = \frac{1}{|T|} * sum_{t \in T} \mathbb{1}(S(x^{(t)}) \neq V(x^{(t)})) \quad (10)$$

Table 1 Characteristics of used datasets

Dataset	Acronyms	#samples	#classes
CIFAR10 [38]	CIF10	60000	10
CIFAR100 [38]	CIF100	700000	100
STL10 [39]	STL10	1300	10
ImageNet [40]	IMN	14197122	1000
MNIST [41]	MNI	70000	10
KMNIST [42]	KMNI	70000	10
Fashion MINIST [43]	FAMNI	70000	10
SVHN [44]	SVH	99289	10
Caltech256 [45]	CAL256	30607	256
GTSRB [46]	GTS	518939	43
U.S. Postal Service [47]	USP	9298	3
Belgium Traffic Signs [48]	BTS	7095	62
Google Speech Commands [49]	GooSP	44000	12
IMDB [50]	IMD	50000	2
Mushrooms [51]	MSHR	8124	2
Adult [51]	ADU	48842	2
Flowers-17 [52]	FLO17	1360	17
Tiny-Images [53]	TIMG	79302017	5
TinyImageNet [54]	TIMGN	10000	200
ImageNet1k [55]	IMG1K	1381167	1000
Indoor67 [19]	IND67	15620	67
CUB200 [38, 56]	CUB200	11788	200
CUBS200 [57]	CU200	11788	200
CINIC10[58]	CIN10	270000	10
CelebFaces Attributes CelebA [59]	CELA	202599	10177
Skin Cancer [60]	SKCR	12500	7
IRS Tax Pattern [61]	IRS	191283	51
General Social Survey [61]	GSS	51020	3
Email Importance [61]	EI	4709	2
Steak Survey [61]	SS	412	5

Table 2 Used attacks and models

Attack	Acronym	Model	Acronym
ActiveThief [8]	ACTH	AllConv [62]	AllCon
CopyCat CNN [63]	CopyC	Convolutional Neural Networks [64]	CNN
DataFree [65]	DataF	DenseNet121 [66]	DenNet121
DFME [65]	DFME	Distribution-aware adversaries [40]	DAA
DaST [15]	DaST	Gaussian mixture models [67]	GMM
Distillation [68]	Dist	Inception [69]	Inc
EntropyRev [29]	EntRev	Knowledge-limited adversaries [45]	CLA
Fine-Tuning [70]	FineT	LeNet [71]	LetNet
Fine-Pruning [59]	FineP	Logistic regression [72]	LR
Jacobian-based DataAugmentation [11]	JBDA	MobileNet [73]	MobNet
JB-self [16]	JBSelf	MobileNetV2 [74]	MobNetV2
JB-top3 [16]	JBTop3	MnistNet [75]	MNet
JBA-PGD [76]	JBPGD	ResNet [77]	ResNet
JBACW_l2 [78]	JBCWL	ResNet18 [79]	ResNet18
JBDA-TR [16]	JBDATR	ResNet50 [80]	ResNet50
knockoff [7]	K.off	ResNet34 [81]	ResNet34
LabelQuery [6]	LabelQ	Recurrent Neural Networks [82]	RNN
Logit-query [6]	LogitQ	VGG16 [83]	VGG16
Modified Architecture [84, 85]	ModArch	VGG19 [86]	VGG19
MixMatch [84]	MixM	Xception [87]	Xcep
Neural cleanse [88]	NEUCL	WideResNet [89]	WRN
PAPERNOT [11]	PAP	WRN-28-10 [90]	WRN-2810
Pruning [59]	PRU	WRN-16-1 [89]	WRN-161
Random query attack [17]	RA	WRN-16-10 [89]	WRN1610
Sample [27]	Sample	3-layer neural network [91]	3LNN
Targeted Randomly Chosen Diraction [16]	TRND		
TRND64 [16]	TRND64		
Tramer attack [6]	TRA		
WorstCase [29]	WorstC		
Zero-shot Learning [92]	ZeroS		

6.3.2 Speed

Speed [16] represents the maximum number of queries that allow the detection of an alert. It has a relation with the amount of information extracted.

6.3.3 False positive rate

In general, False Positive Rate (FPR) represents the rate of negative samples that are misclassified. In [16], FPR represents the ratio of false alarms raised by a detection method to all query sequences from benign clients. In [18], FPR represents the percentage of benign queries raising alarms.

6.3.4 Detection accuracy DAcc

DAcc is a measure of accuracy which is used to detect malicious queries within the MLaaS system. It is used by the $FDIN_{ET}$ approach [21].

6.3.5 Colluding detection accuracy

Colluding Detection Accuracy (CDAcc) represents the proportion of the colluding adversaries among N clients in MLaaS. It is defined in the Equation 11:

$$CDAcc = \frac{\sum_{u<v\in[1,N]} \mathbb{1}[FDIN_{ET}(u,v) = J(u,v)]}{C_N^2} * 100 \tag{11}$$

Where:

- u and v are two clients (among N clients).
- J(u,v): is a judgment function that returns 1 if u and v are colluding adversaries.
- C: is combination formula.

6.3.6 Extraction status

Extraction Status (ES) [21] is used to detect extraction queries. It is introduced by EM (see subsection 4.1.1) that uses a local proxy to monitor the information gain of each user. The information gain serves to quantify the level of model privacy leakage from the victim model. ES is defined in the Equation 12.

$$ES = \frac{100}{|D_{test}|} * \sum_{x \in D_{test}} \mathbb{1}[F_V(x) = F_{V'(x)}] \tag{12}$$

Where D_{test} is the test dataset.

A lower ES is preferable for benign queries, while higher ES is better for malicious queries.

6.3.7 Agreement

Agreement (Agr) [16] represents the accuracy of the surrogate model S, by taking the target model M prediction as ground truth.

6.3.8 Extraction rate

Extraction rate (ER) [6, 17] is used to measure the proportion of matching predictions between the victim model F and the surrogate model S for the set of queries. It is defined in the Equation 13:

$$ER = \frac{1}{|X_e|} * \sum_{x_i \in X_e} \mathbb{1}(F(x_i) = S(x_i)) \tag{13}$$

Where:

- X_e: is the set of queries.
- $\mathbb{1}(\bullet)$ is an indicator function. It is equal to 1, if F and S gave the same label, 0 otherwise.

6.3.9 Utility

Utility (U) measures the proportion of perturbed responses provided by BDPL method. It is defined in the Equation 14

$$U = \frac{1}{|X_q|} * \sum_{x_i \in X_q, y_i \in Y_q} \mathbb{1}(f(x_i) = y_i) \tag{14}$$

Where:

- X_q: is the set of queries sent by a client.
- Y_q: is the set of perturbed responses.

6.3.10 Watermark success rate

Watermark success rate ($SRate_{wm}$) is used by EWE method [5]. It represents the probability of correctly identifying watermarked (to class c_t).

6.3.11 Test accuracy and watermark accuracy

These two measures are used by DAWN method [32], in order to evaluate the accuracy of the surrogate model S for test dataset and for trigger dataset.

Test accuracy Acc_{test} represents the proportion of well classified samples of test dataset by S. It is defined in the Equation 15.

$$Acc_{test}(S) = \frac{1}{|test|} * \sum_{x_i \in test} \begin{cases} 1 & \text{if } S(x_i) = O_f(x_i) \\ 0 & \text{o } therwise \end{cases} \tag{15}$$

Watermark accuracy Acc_{wm} represents the proportion of well classified samples of trigger dataset TA. It is defined in the Equation 16.

$$Acc_{wm}(S) = \frac{1}{|T_A|} * \sum_{x_i \in T_A} \begin{cases} 1 & \text{if } S(x_i) = B_v(x_i) \\ 0 & \text{o } therwise \end{cases} \tag{16}$$

Where: Bis the backdoor function.

DAWN aims to minimize Acc_{test} and maximize Acc_{wm}. However, an adversary aims to maximize Acc_{test}, and minimize Acc_{wm}.

6.3.12 Confidence score and p-value

So that dataset inference methods can conclude that a model is stolen or not, the pair-wise [22] is applied to compute the values of p-value and the confidence score $\Delta\mu = \mu_S - \mu_B$. Where μ_S and μ_B are mean confidence score of the confidence score vectors S and B, respectively. These vectors are obtained by querying models respectively from public and private datasets, respectively.

If p-value is smaller than the significance level α, then the null hypothesis H_0 is rejected, and we conclude that the model is stolen. The confidence score $\Delta\mu$ represents the verification confidence, the larger $\Delta\mu$ implies that the verification is more confident.

6.4 Results obtained by state of the art defense methods

In order to show the robustness of defense methods, we present at first the used tools. Then, we present the obtained results followed by an analysis and a discussion.

In the Tables 3, 4 and 5, we present the used tools for passive, active, and reactive and proactive defences respectively. Each table contains datasets, victim models, tested attacks, parameters and evaluation measures used by each method.

In the rest, we use these acronyms:

- #Acs: number of fake accounts.
- eAcc: extraction accuracy.
- Acc: accuracy.
- ErrCl (D): defender's classification error.
- ErrCl(A): adversary's classification error.
- Acc (D), Acc (C), and Acc(S): design defender Accuracy, clone Accuracy, and Accuracy of the stolen model respectively.

6.4.1 Results obtained by passive defenses

Results obtained by EM: To evaluate the performances of EM, the measures ES and 1-R_{test} are computed by varying the number of queries of both strategy 1 and strategy 2. Each strategy is tested when an adversary launches Tramer attack and random queries for tested dataset. The obtained results are illustrated in the Fig. 2. As the strategy 2 is more appropriate for continues features, the results of this strategy is missed for SS dataset.

The model extraction status for colluding adversaries is determinate by following the same steps for 10 clients, in which k clients are colluding, where the values of k are 1, 3, and 5. The obtained results are illustrated in the Fig. 3.

As shown in the Figure 2, we remark:

- 1-R_{test} increases when the number of queries increases in the case of Tramer attack to recover the rule sets of all leaf nodes (noted User RS (Tramer) in the Fig. 2).
- 1-R_{test} varies according to the dataset and the distribution of random queries (refers to Monitor User DT (Rand) in the Fig. 2) when an adversary learns the decision tree from random queries.
- 1-R_{test} of the decision tree constructed by the monitor takes sometimes a lower value, for the strategy 1 and Tramer attack (refers to Monitor DT (Tramer) in the Fig. 2), because the training set and the queries generated by Tramer have a different distribution.
- ES matches 1-R_{test} of the adversary's model based on Tramer attack ("User RS(Tramer)") for the strategy 1.
- ES is more closely matches the 1-R_{test} for both Tramer attack and random queries for strategy 2, because the monitor conserves model summaries based on the span of the future space, which is less affected by the distributions of queries.

We conclude that the accuracy (1-R_{test}) of the surrogate model depends on the number of queries, the distribution of the training set and queries. Also, the information gained increases when the number of colluding users increases. For example, ES for GSS dataset for k = 1 is 28%, for k = 3 is 51%, and for k = 5 is 69% (Fig. 3)

Results obtained by PRADA:

Table 3 The different tools used by passive defenses

	Datasets	Victim models	Attacks	Parameters	Measures
EM	IRS, GSS, EI, SS	Decision tree	TRA, RA	Threshold t	1-R_{test}, ES
PRADA	MNI, GTS		TRA, PAP	δ	Speed, FPR
	For demonstration: SPS, BTS		TRND, COLOR		
	randomly generated samples				
SEAT	CIF10, CIN10, TIMGN	VGG16	JBTop3, JBPGD	δ (use binary search), Nhresh=50	FPR, #Acs, eAcc
	IMG1K, CIF100, CAL256		JBCWL, RS		
	IND67, CU200, SVH				
$FDIN_{ET}$	CIF10, GTS, CELA, SKCR	VGG19, MobNetV2	JBDA, TRND, K.off, DFME, DaST, ACTH	K=10, 100	FPR, DAcc, ES, CDAcc
		DenNet121, ResNet50		L= 5 layers	
				bs=50, 500	
				ι_1	
				ι_2=0.05	

Table 4 The different tools used by active defenses methods

	Datasets	Victim models	Attacks	Parameters	Measures
AM	**D** $_{in}$:MNI, FAMNI,CIF10, FLO17. **Outliers:** KMNI, KMNI, IMH1K, IND67	LeNet, ResNet18	K.off, JBDA	/	Acc(D), Acc(C)
BDPL	MSHR, ADU	**Linear model:** LR, 3-layer neural **Non-linear model:** 3-LNN	Fine-tuned queries	Δ, ϵ	U, ER
RSP	MNI, CIF10, CIF100, STL10, IMD, FAMNI	MobNet, Xcep, AllCon	Sample (the best attack selected)	/	Agr
MAD	MNI, FAMNI, CIF10, CIF100, CUB200, CAL256	VGG16, LeNet	JBDA, JBSelf, JBTop3, k.off	ϵ	Acc(S)
GRAD	CIF10,CIF100, CUB200,CAL256	ResNet50, 40-2 Wide ResNets	CLA, DAA	ErrCl(D), l1 distance	ErrCl (A)
EDM	**D** $_{in}$ **datasets:** MNI,FAMNI,CIF10, CIF100,FLO17. **D** $_{out}$ **datasets:** KMNI, MNI, TIMG, IMG1K	Conv3, Wres16-4, Wres16-4, ResNet18	K.off, JBDA, JBDATR	05 models, M=10, m=2	Acc

Table 5 The different tools used by reactive and proactive defenses

	Datasets	Victim models	Attacks	Parameters	Measures
Reactive defenses					
Watermarking					
EWE	MNI, FAMNI, GooSP	CNN, RNN	PRU, FineP, NEUCL	W, T, (c_s,c_t)	$SRate_{wm}$, Acc
DAWN	KMNI, GTS, CIF10, CAL256	MNIST-5 L, GTSRB-5 L, CIFAR10-9 L, GTSRB-RN34, CIFAR10-RN34, Caltech-RN34	K.off, TRND64	e=50%	Acc_{test}, Acc_{wm}
Dataset inference					
DI-OR	CIF10, CIF100	WRN, WRN2810, WRN161, WRN1610	ModArch, Dist, ZeroS, FineT, LabelQ, LogitQ	α=0.05	p-value, $\Delta\mu$
DI-SSM	CIF10, SVHN,IMN	GMM	ResNet18, ResNet34,3x3 Conv layer of stride 1, ResNet50	α=0.05	p-value, $\Delta\mu$
DI-EEF	CIF10, IMN (subset)	WRN, ResNet	Dist, ZeroS, FineT, LabelQ, LogitQ	α=0.01	p-value, $\Delta\mu$
Proactive defenses					
PoW-based defense	KMNI, FAMNI, SVHN, CIF10	MNET, ResNet34	JBDA, JBDATR, DataF, MixM, CopyC,EntRev, WorstC	/	Privacy cost, Execution time

To evaluate FPR, PRADA uses natural data from MNI and GTS datasets, and it may use these datasets: randomly generated samples, SPS, and BTS in order to demonstrate that PRADA is independent of a specific data distribution, because SPS and BTS contain similar data as MNI and GTS with different distribution. Since FPR is computed for benign clients, PRADA simulates a benign client by randomly picking 6000 samples from a given dataset and successively querying the appropriate model: MNI/USP/random $\longrightarrow$ MNI model, GTS/BTS/random $\longrightarrow$ GTS model. To evaluate FPR, the sequence of queries is spitted into 120 chunks of 50 queries each and counts a false positive if at least one alert is triggered by PRADA in a chunk.The results obtained are illustrated in the Fig. 4:

In the Fig. 4, we remark that FPR depends on the value of δ, and it increases when δ increases. A high δ value of 0.96 results in nofalse positives for MNI model while GTS model requires $\delta = 0.87$ to reach the same result. We

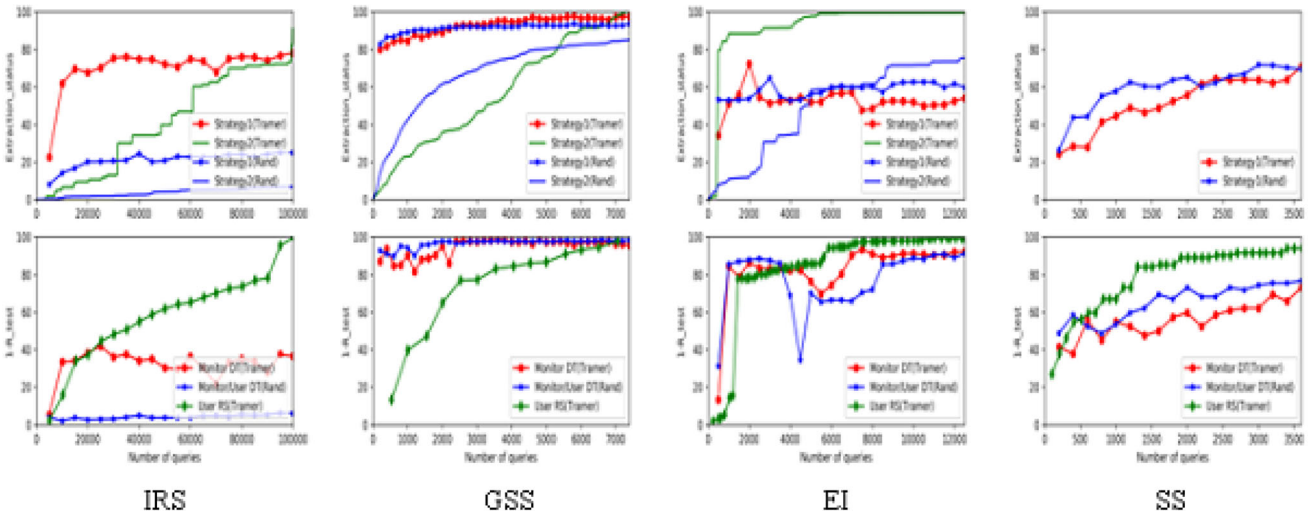


Fig. 2 ES and 1-R_{test} obtained by EM

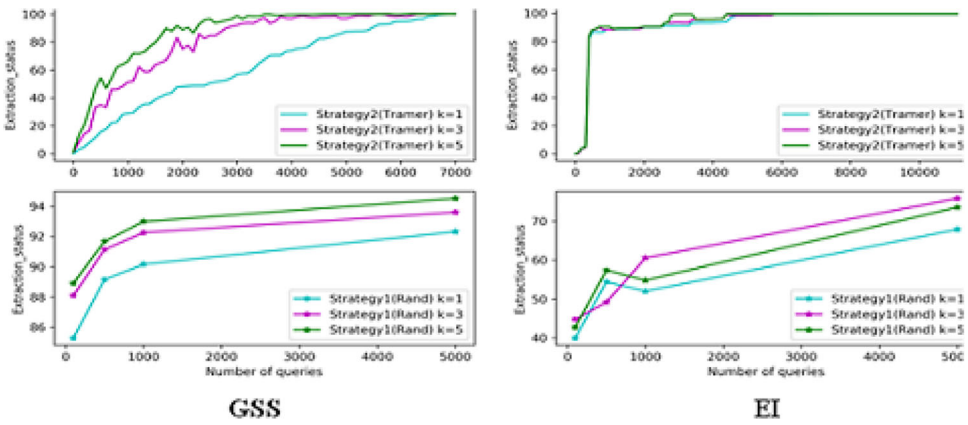


Fig. 3 Results of model extraction status for colluding adversaries for GSS and EI datasets

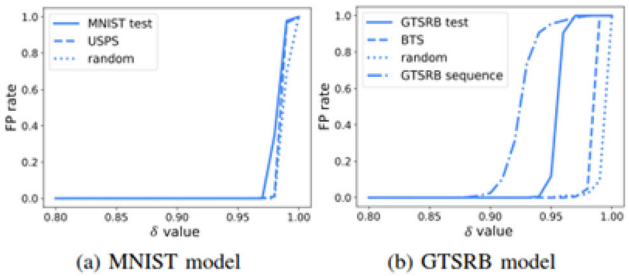


Fig. 4 FPR for PRADA vs. detection threshold

remark also that FPR depends on the distribution of queries:

- If queries are distributed close to normal then there is no false positive if δ is high. It is the case of BTS and random queries.
- If queries are further away from a normal distribution then there is no false positive if δ is low. It is the case of GTS.

According to the Table 6, the attacks PAP, TRND and COLOR are rapidly detected after they have a change in their strategy (from natural to synthetic samples). For TRA attack, the detection is slower because it is itself slow extracting DNNs.

Results obtained by SEAT: For different attacks, SEAT may determine the impact of size of seed query set on the

Table 6 Speed of PRADA using different attacks and different values of δ

Model	δ	Speed			
		TRA	PAP	TRND	COLOR
MNIST	0.95	5560	120	140	-
MNIST	0.96	5560	120	130	-
GTS	0.87	5020	430	Missed	550
GTSRB	0.90	5020	430	Missed	480
GTSRB	0.94	5020	430	440	440

Acc. To do, it runs random sampling (RS) attack as the baseline attack, and test these values 500, 5000, and 50000 as a size of seed query set (which are 1%, 10%, and 100% of the query budget respectively). The best result is obtained by JBPGD; because it needs only 500 seed samples to achieve the highest extraction accuracy gain which is 7%.

To detect the strongest detector, SEAT trains four similarity encoders to determine the best. The best similarity decoder is the one that cancels the maximum number of victim's accounts.

The first three are trained by using adversarial training with PGD attack with 03 these values of random:$\epsilon = 8/255$, 16/255, or 24/255.The fourth one is trained using random transformations (RT). According to the results obtained in the Table 7, the strongest detector is JBPDG with ϵ =8/255.

Table 8 shows that FPR of SEAT detector is very low when benign users send queries from different query sets, and its value is under 0.05% for all datasets.

To prove the effectiveness of SEAT detector, the number of fake accounts and the extraction accuracy for different variants of JBA attackers are computed.

According to the results shown in the Table 9, the worst results are obtained by JBTOP3 and JBACWL, because the number of fake accounts is the highest and the extraction accuracy is the lowest. However, the best results are obtained by JBPGD attacks and they depend on the value of ϵ, where the number of account is the lowest and the accuracy is the highest for ϵ =24/255.

Results obtained by FDINET:

$FDIN_{ET}$ is evaluated using several metrics. In the following subsections, we present the different obtained results.

Detection accuracy and FPR

$FDIN_{ET}$ tests the different values of bs and τ_1 to determine the values of DAcc and FPR. The obtained results as presented in the Table 10. Specifically, $FDIN_{ET}$ achieves a DAcc. of 100% and an FPR close to 0.0 for DFME and DaST with a batch size (bs) of 500. Also, the FPR is 0 or close to 0 and the DAcc is high for only 50 queries. However, DAcc is lower for CIF10 dataset for K.off and ACTH. We conclude that $FDIN_{ET}$ is capable of identifying malicious clients.

As $FDIN_{ET}$ doesn't use a proxy model, it computes ES as the confidence score (AC *100)%.For benign clients, ES is below 34.60% for CIF10, 44.20% for GTS, 17.40% for CELA, and 9.30% for SKCR. For attacks: JBA, T-RND, k.off, ACTH, DFME, and DaST, the ES is an average of 96.08% and 84.94% for GTS and SKCR, respectively. Therefore, $FDIN_{ET}$ is able to identify malicious clients.

To detect colluding adversaries, 100 clients are simulated, each submitting 50000 queries, with bs=500. As an example, we show the confusion matrix p-value for CIF10 dataset in the Fig. 5.

According to the obtained results shown in the Fig. 5, we conclude: $FDIN_{ET}$ is not efficient to distinguish between k.off and ACTCH, because p-value (ACTH, k.off) =0.46, and p-value(k.off, ACTH)=0.76. However, it is efficient for identification of the colluding adversaries, because p-value of the diagonal of the confusion matrix and the remaining combinations is high, and the obtained DAcc is greater than 91%.

Table 7 Detection of the best detector

JBAPGD ϵ	RT	SEAT with ϵ		
3-5		8/255	16/255	24/255
8/255	13	65	43	28
16/255	9	52	32	22
24/255	7	41	25	18
32/255	6	33	20	15
40/255	4	28	17	12

Table 8 FPR of SEAT detector

Query set	FPR (%)
CIF10	0.012
TIMGN	0.007
IMG1K	0.010
CIF100	0.013
SVH	0.026
CIN10	0.009
IND67	0.006
CU200	0.017
CAL256	0.027

Table 9 #Acs and eAcc for JBPGD attack (ϵ = 24/255)

Seed set	#Acs	eAcc(%)
JBTop3	122	69
JBACWL	109	62
JBPGD(ϵ =8/255)	65	73
JBPGD (ϵ =16/255)	52	74
JBPGD (ϵ =24/255)	41	75
JBPGD (ϵ =32/255)	33	72
JBPGD (ϵ =40/255)	28	71

6.4.2 Results obtained by active defenses

Results obtained by AM: AM evaluates the Acc (D) vs Acc (C) as it is shown in the Fig. 6.

According to the Fig. 7, the evaluations against k.off and JBDA attacks show that AM degrades Acc (A) by up to 40% with a minimal impact on the Acc (D) (< 0.5%). Therefore, AM is considered as an accuracy-preserving defense.

Results obtained by BDPL: To evaluate BDPL, the values of ER and U are computed for both BDPL and no defense. This is done by varying the number of queries of all possible combinations, and they are compared, with $\Delta = 1/8$ and $\epsilon = 0.01$. The obtained results are presented in the Fig. 7:

Figure 7 shows that there is a signification difference in ES between BDPL and no defense (up to 12% drop on R) for both linear and nonlinear models. Also, we observe that U saturates at over 80% after 20K queries in all combinations except for Adult w/ Logistic Regression. We conclude that BDPL can protect efficiently the decision boundary.

Results obtained by RSP: As the objectives of RSP are the prevention of model replication, and the retaining the performance of the protected model, then RSP determines the agreement of the protected model and that of stolen model. We consider the following notations:

- S(M, Reverse sigmoid): is the agreement of the stolen model.
- P(M, Reverse sigmoid): is the agreement of the protected model.

RSP evaluates the two previous measures on tested datasets. The results are shown in the Table 11.

In the Table 11, we observe that the agreements for the protected models still high. However, the agreement for the stolen model is dropped more than 20% for all tested datasets. These results prove the effectiveness of RSP.

Results obtained by MAD: In MAD, the defenses are evaluated on a nonreplicability vs. utility curve at various operating points ϵ of the defense, for a large query budget (50K). The obtained results are shown in the Fig. 8. To successfully mitigate attacks as a defender, MAD wants the defense curves (colored solid lines with operating points denoted by thin crosses) to move away from undefended accuracies (denoted by circular discs, where $\epsilon = 0.0$) to ideal defense performances (cyan cross, where Acc (D) is unchanged and Acc(Attacker) is chance-level).

We observe from Fig. 8 that the effectiveness of the attacker can be greatly reduced. Across all models, MAD provides reasonable operating points (above the diagonal), where defender achieves significantly higher test accuracies compared to the attacker.For instance: - For MNI dataset: the accuracy of JBTop3 is reduced from 87.3% to 35.7%, and that of k.off is reduced from 99.1% to 69%. While for victim model, it drops by a very small value (< 1%). - For CUB200 dataset, the accuracy of k.off is

Table 10 The DAcc, and FPR of $FDIN_{ET}$

	JBDA	TRND	K.off	ACTH	DFME	DaST
CIF10, bs=50,ι_1=0.47						
DAcc	89.90	93.10	80.26	82.00	100	98.03
FPR	4.10	2.60	4.10	2.90	1.70	1.80
CIF10, bs=500,ι_1=0.48						
DAcc	91.00	93.00	90.00	92.00	100.0	100.0
FPR	0.00	0.0	0.0	0.0	0.0	0.0
GTS, bs=50,ι_1=0.77						
DAcc	90.30	90.40	100	95.60	100	100
FPR	0.77	0.0	0.0	0.0	0.0	0.0
GTS, bs=500,ι_1=0.78						
DAcc	88.0	87	100	95	100	100
FPR	0.0	0.0	0.0	0.0	0.0	0.0
CELA, bs=50,ι_1=0.33						
DAcc	96.61	97.10	83.93	97.10	100	100
FPR	0.30	0.10	0.30	0	0.1	0.2
CELA, bs=500,ι_1=0.36						
DAcc	95.00	96.0	93.00	93.00	100	100
FPR	0.0	0.0	0.0	0.0	0.0	0.0
SKCR, bs=50,ι_1=0.51						
DAcc	92.80	93.20	88.70	89.90	99.80	97.40
FPR	0.0	0.0	0.0	0.0	0.0	0.0
SKCR, bs=500,ι_1=0.48						
DAcc	94.00	94.00	100.0	80.0	100	100
FPR	0.0	0.0	0.0	0.0	0.0	0.0

Fig. 5 Confusion matrix for average hypothesis tests' p-values over different clients for CIF10

	JBA	TRND	K.off	ACTH	DFME	DaST	Benign
Benign	1.89e-6	1.03e-18	0.000573	30.00976	4.73e-12	2.01e-9	0.929
DaST	0.0245	0.00873	0.0356	1.09e-6	0.0551	0.987	0.00052
DFME	0.00731	1.72e-7	6.92e-12	0.0424	0.985	0.063	2.12e-12
ACTH	3.6e-8	0.0178	**0.736**	0.0691	1.05e-21	5.93e-6	1.89e-5
K.off	4.63e-6	2.15e-21	0.988	**0.46**	6.03e-8	3.88e-9	5.62e-12
TRND	0.086	0.977	2.69e-11	0.000567	0.00779	0.00562	1.16e-9
JBA	0.99	1e-11	0.0157	0.00728	0.00349	2.48e-12	1.18e-7

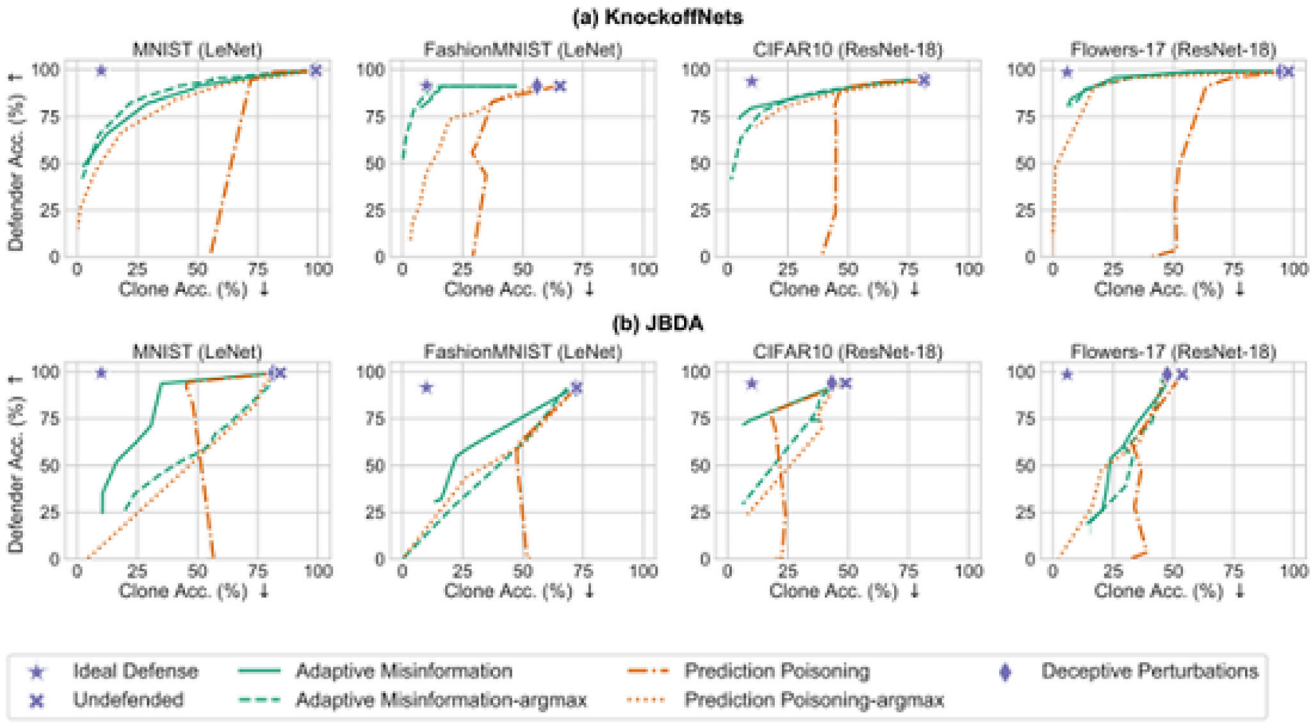


Fig. 6 Acc (D) vs Acc (A) for AM

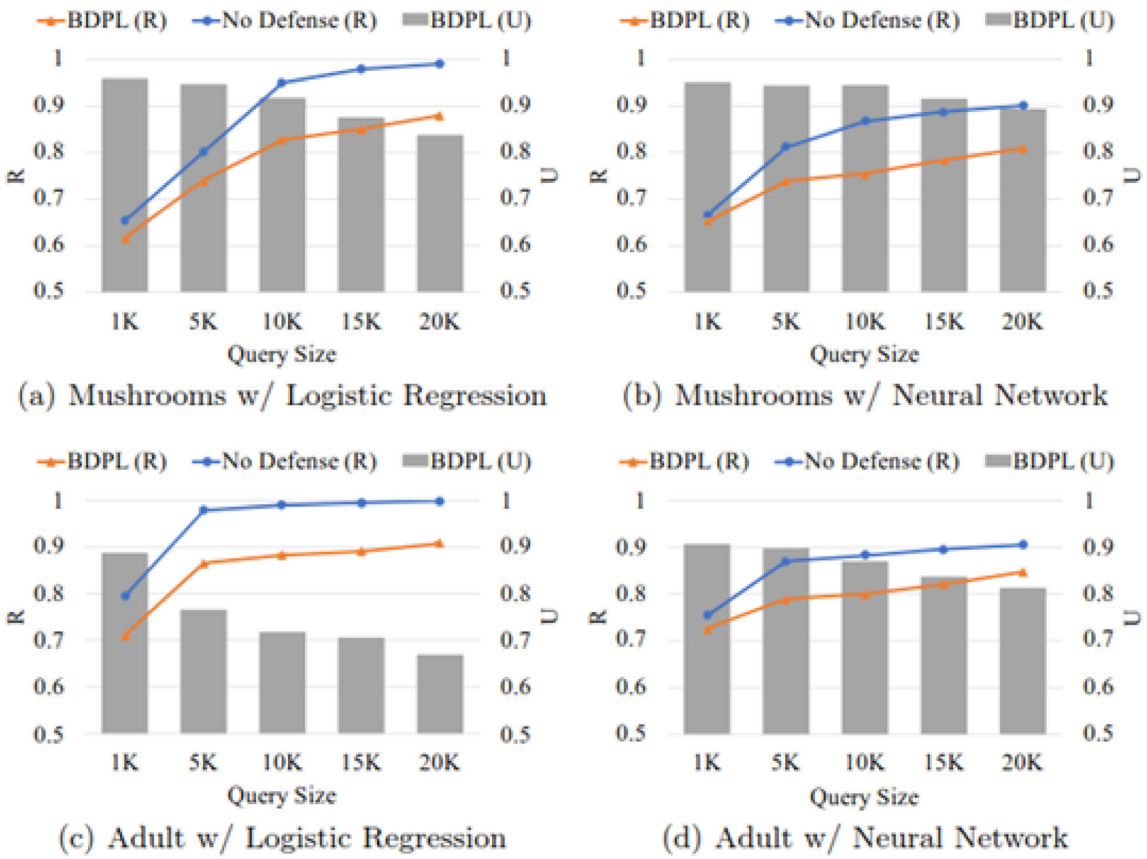


Fig. 7 Overall Protection Effect by BDPL: ER and U

Table 11 Results obtained by RSP

	MNI	FAMNI	CIF10	CIF100	STL10	IMD
P(M, Reverse sigmoid)	0.99	0.99	0.85	1	0.99	1
S(M, Reverse sigmoid)	0.68	0.79	0.40	0.02	0.37	0.60

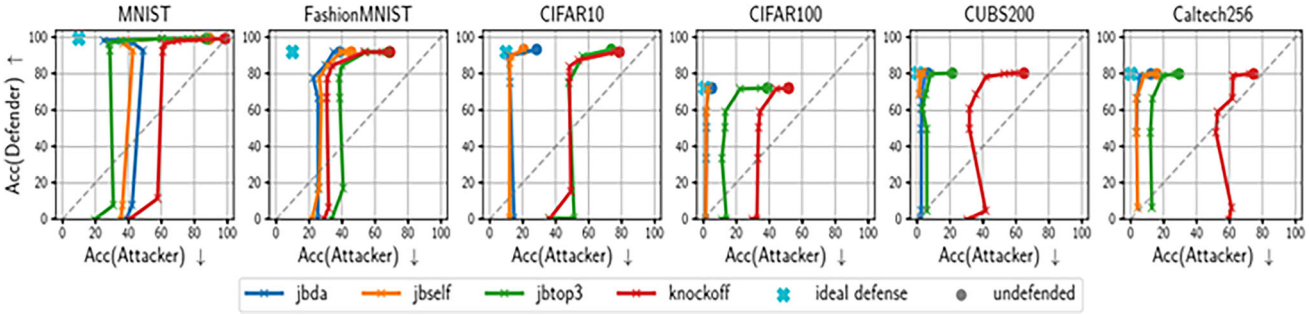


Fig. 8 Attackers vs. MAD

reduced from 65.1% to 41.9%. While for victim model, it drops by 2%.

Consequently, we conclude that MAD is an effective approach to defend against several attacks, because it decreases the accuracy of the stolen model, without a significant difference of the accuracy of the victim model.

Results obtained by GRAD:

GRAD explores knowledge-limited and distribution-aware adversaries for these evaluation datasets: CIF10, CIF100, and CUB200. For distribution-aware adversaries, it constructs query sets from ImageNet-1K by manually selecting overlapping classes. This gives ImageNetC10, ImageNetC100, and ImageNetCUB200, which are paired with their matching evaluation set. Knowledgelimited adversaries query sets for the above evaluation datasets are CIF100, CIF10, and Cal256 respectively.

GRAD evaluates its effectiveness against an adversary by computing the ErrCl (A). To do, it takes into consideration the utility metrics: Err Cl(D) and l1 distance by testing 03 values of each metric. Also, it uses transfer and evaluation data, which is noted transfer $\longrightarrow$ evaluation. The obtained results are shown in the Table 12.

According to the obtained results (shown in the Table 12), we conclude that GRAD is a strong defense, because the classification error of an adversary is high for both CLA and DAA query sets. Also, we remark that ErrCl(A) increases where l1 distance increases, because the information is destroyed.

Results obtained by EDM:

EDM uses an ensemble of 5 models that are trained on the accuracy and diversity objectives. It requires an auxiliary Out Of Distribution data (refers as D_{out} in the Table 5) for diversity objective. It evaluates its robustness by comparing the clone accuracy achieved by various attacks on EDM and an undefended baseline model, and it takes into consideration two cases of returned prediction: soft-label prediction and hard label prediction. In the first case, a probability distribution over the output classes is returned. However, in the second case, the argmax class-label is returned. The obtained results are represented in the Table 13

According to the Table 13, EDM can reduce the accuracy of the tested attacks offering up to a 39.7% reduction in clone accuracy compared to an undefended baseline model. Therefore, we conclude that EDM is effective against attacks.

6.4.3 Outcomes attained by proactive defenses

Privacy cost of PoW is computed by using PATE. Figure 11 shows the privacy cost for legitimate user (refers to the standard) and different attacks by varying the number of queries. We observe that the privacy cost increases much more for attacks that use Out Of Distribution data (DataF, CopyC, and k.off) compared to the attacks that use In Of Distribution Data.

6.4.4 Results obtained by post stealing defenses

Results obtained by Watermarking methods: In the table 14, we present results obtained by EWE and DAWN methods at first.

As shown in the Table 14 in the case of EWE, accuracy and watermark success rate of the baseline victim model are near 100% for all tested datasets. However, the watermark success rate for extracted model quickly drops to near-zero on both MNI and FAMNI for the extracted model, and near 20% for GooSP.

As shown in Table 14, the extracted EWE model averages at least 39% higher watermark success rate than

Table 12 Classification error of an adversary (ErrCl (D)) obtained by GRAD

	ErrCl(D)			l1 distance		
	1	2	5	0.1	0.2	0.5
CLA: ErrCl(A)						
ImageNet-C10 ⟶ CIF10	16.4	21.5	23.4	9.1	10.1	15.5
ImageNet-C100 ⟶ CIF100	43.4	47.6	53.0	36.5	7.7	44.1
ImageNet-CUB200 ⟶ CUB200	54.1	56.4	60.7	41.8	44.6	55.6
DAA:ErrCl(A)						
CIF100 ⟶ CIF10	17.5	20.5	22.4	10.6	11.7	17.0
CIF10 ⟶ CIF100	55.2	59.3	63.7	46.3	48.0	56.8
CAL256 ⟶ CUB200	57.9	60.7	65.2	42.5	46.1	58.3

Table 13 Results obtained by EDM

	MNI	FAMNI	CIF10	CIF100	FLO 17
JBDA for soft Label					
Acc. Undefended model	88.8	73.28	22.03	4.09	11.76
Acc. EDM	79.04	72.34	22.01	3.69	11.40
JBDA for hard label					
Acc. Undefended model	81.53	70.97	18.99	4.23	11.76
Acc. EDM	72.64	66.50	20.08	3.75	11.03
JBDATR for soft Label					
Acc. Undefended model	89.82	73.27	21.44	4.45	20.59
Acc. EDM	73.08	74.17	20.96	4.22	18.75
JBDATR for hard Label					
Acc. Undefended model	80.62	70.23	24.47	5.24	15.07
Acc. EDM	73.18	69.46	18.78	3.38	16.54
K.off for soft Label					
Acc. Undefended model	90.18	34.76	85.39	53.04	69.85
Acc. EDM	51.34	15.28	68.50	41.16	30.15
K.off for Hard Label					
Acc. Undefended model	61.20	28.60	73.64	28.50	56.25
Acc. EDM	41.25	15.64	54.98	24.69	28.68

in the baseline extraction case, indicating better extraction survivability.

DAWN proves the ownership of a model by computing the value of Acc_{wm}, and comparing it with the value of tolerated error e: if Acc_{wm} is larger than 1-e then the model is stolen.

As shown in the Table 14, DAWN can prove the ownership of a model because $Acc_{wm} > 0.5$ for both TRND64 and k.off attacks. We observe also that DAWN decreases the Acc_{test} of tested models for both TRND64 and k.off attacks for all tested datasets.

EWN is evaluated against the following backdoors: PRU, FineP, and NEUCL in order to determine the values of Acc and $Rate_{wm}$. Figures 9 and 10 show the obtained results for both PRU and FineP.

In Fig. 9, watermark success rate remains high despite significant pruning. In fact, the watermark success rate only starts decreasing below 40% when the model's accuracy on task data also significantly decreases (below 60%). Such a model becomes effectively useless to the adversary, who would be better off training a model from scratch. We conclude pruning is ineffective against EWE.

According to the Fig. 11, we observe that in the most favorable setting for FineP, watermark success rate on the extracted model remains around 20%, which is still enough to claim ownership. This is despite the fact that 50% of the architecture's neurons were pruned. Because data used for fine-tuning is labeled by the watermarked victim model, it contains information about the watermarks even when the labels provided by the watermarked model are for task data.

Table 14 Results obtained by watermark methods

Datasets	MNI		FAMNI		GooSP	
EWE						
Victim model	Baseline	EWE	Baseline	EWE	Baseline	EWE
Acc (%)	98.28	97.75	90.22	90.42	97.00	96.19
Rate $_{wm}$(%)	100	99.85	100	99.88	100	100
Extracted model	Baseline	EWE	Baseline	EWE	Baseline	EWE
Acc	98.35	97.58	89.43	89.45	96.78	96.65
Rate $_{wm}$(%)	3.09	61.44	5.75	44.90	22.58	86.16
DAWN						
TRND64						
Models	MNIST-5 L		GTSRB-5 L		CIFAR10-9 L	
	F_v	F_s	F_v	F_s	F_v	F_s
Baseline Acc $_{test}$(%)	98.71	95	91.50	61	84.53	60.03
F_s with DAWN	Acc_{test}(%)	Acc_{wm}(%)	Acc_{test}(%)	Acc_{wm}(%)	Acc_{test}(%)	Acc_{wm}(%)
	78.93	100	61.43	98.23	60.95	71.17
K.off						
Models	GTSRB-RN34		CIFAR-10		CALTECH-RN34	
	F_v	F_s	F_v	F_s	F_v	F_s
Baseline Acc $_{test}$(%)	98.42	97.43	94.66	88.27	67.62	72.74
F_s with DAWN	Acc_{test}(%)	Acc_{wm}(%)	Acc_{test}(%)	Acc_{wm}(%)	Acc_{test}(%)	Acc_{wm}(%)
	97.72	100	88.41	72.54	71.98	93.54

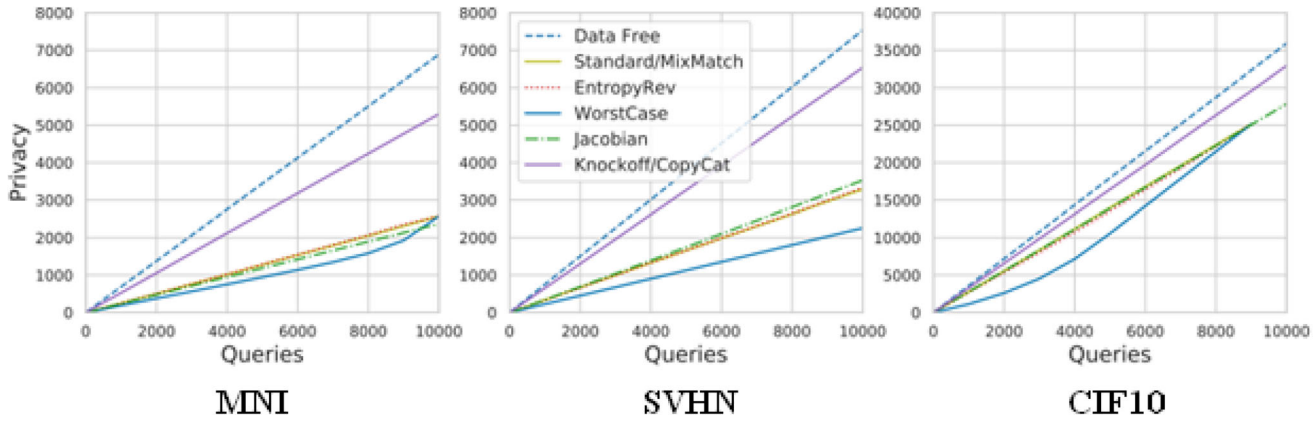


Fig. 9 Privacy cost vs number of queries

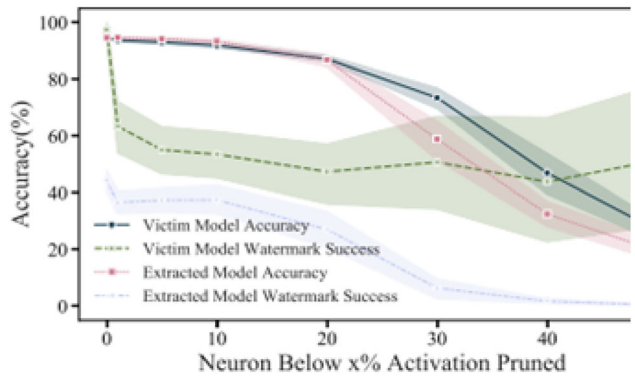


Fig. 10 Acc and Rate$_{wm}$ on the extracted model in the face of a PRU attack

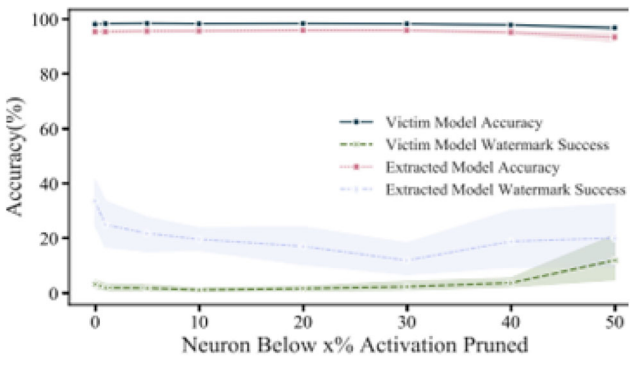


Fig. 11 Acc and Rate$_{wm}$ on the extracted model in the face of a FineP attack

NEUCL defines a model as backdoored if an Anomaly Index AI derived from the analysis is above a certain threshold (set to 2). For more details, see [69]. AI (baseline model) =31.6, and AI (EWE) = 1.6. Therefore, NEUCL is unable to identify our watermark and its trigger. Consequently, it is ineffective against EWE.

Results obtained by dataset inference methods:

To prove the robustness of each state of the art dataset inference method, different attacks are tested against that method to compute pvalue and confidence score $\Delta\mu$ (see details in subsection 6.3.12). For DI-OR method, both MinGD and blind Walk settings are tested. Also, p-values and $\Delta\mu$ are added for the baseline models: 'Source' (where the victim model V is stolen) and 'independent' (the case where the model I is trained on the separate datasets). These values serve as bounds to interpret the obtained results. The obtained results are presented in the table 15.

According to the results shown in the Table 15, pvalue is smaller than α of for the tested dataset inference methods, and $\Delta\mu$ is largest. We conclude that these methods are robust to the model stealing attacks

Table 15 Results obtained by watermark methods

DI-OR								
Method	MinGD		Blind Walk		MinGD		Blind Walk	
Datasets	CIF10				CIF100			
Measure	$\Delta\mu$	p-value	$\Delta\mu$	p-value	$\Delta\mu$	p-value	$\Delta\mu$	p-value
V	0.838	10^{-4}	1.823	10^{-42}	1.219	10^{-16}	1.967	10^{-44}
ModArch	0.568	10^{-4}	0.778	10^{-5}	0.362	10^{-2}	1.098	10^{-5}
Dist	0.645	10^{-4}	1.400	10^{-10}	1.016	10^{-6}	1.471	10^{-14}
ZeroS	0.371	10^{-2}	0.406	10^{-2}	0.466	10^{-2}	0.405	10^{-2}
FineT	0.832	10^{-5}	1.839	10^{-27}	1.047	10^{-7}	1.423	10^{-10}
LabelQ	0.475	10^{-3}	1.006	10^{-4}	0.270	10^{-2}	0.107	10^{-1}
LogitQ	0.563	10^{-3}	1.048	10^{-4}	0.385	10^{-2}	0.184	10^{-1}
I	0.103	1	−0.397	0.675	−0.242	0.545	−1.793	1

DI-EEF				
Datasets	CIF10		IMN	
Measure	$\Delta\mu$	p-value	$\Delta\mu$	p-value
V	0.97	10^{-7}	0.90	10^{-5}
Dist	0.53	10^{-7}	0.61	10^{-5}
ZeroS	0.52	10^{-5}	0.53	10^{-4}
FineT	0.50	10^{-6}	0.60	10^{-5}
LabelQ	0.52	10^{-4}	0.55	10^{-3}
LogitQ	0.54	10^{-4}	0.55	10^{-4}
I	0.00	1.00	10^{-5}	0.99

DI-SSM								
Datasets	CIF10		CIF100		IMN		SVHN	
Measure	$\Delta\mu$	p-value	$\Delta\mu$	p-value	$\Delta\mu$	p-value	$\Delta\mu$	p-value
SVHN	3.04	3.97e−2	13.36	6.35e−41	4.04	3.33e−4	13.36	6.35e−41
CIF10	5.09	8.73e−7	4.61	2.38e−4	6.21	1.47e−4	4.61	2.38e−4
STL10	3.42	1.04e−2	5.22	1.23e−5	5.87	1.09e−4	5.22	1.23e−5
IMN	3.47	6.34e−3	3.74	9.81e−3	7.32	3.14e−5	3.74	9.81e−3

7 Conclusion

In this paper, we have addressed the stealing attack problem, in which we have presented this problem, and the different strategies used by an attacker to steal a model. As these models need protection, several defenses are proposed, and they are categorized in four categories: passive, active, reactive, and proactive defenses. When we have described several approaches of each category, we have concluded that these approaches differ in the way to deal with the studied problem. For instance, active learning defenses aims to decrease the accuracy of the surrogate model by perturbing the predictions, and watermarking techniques are used only to prove the ownership of the model without perturbation.

Experimental studies show the robustness and effectiveness of the proposed approaches, after they have been tested using several datasets and different attacks to compute the appropriate evaluation metrics.

Future research on safeguarding MLaaS against model theft efforts should concentrate on several crucial aspects. Sophisticated detection methods such as real-time surveillance and behavioral analysis can help identify suspicious activity. Improved response techniques, like automated systems and incident response plans, are crucial to reducing risks. Further study is required on preventive techniques like adaptive query restriction and strong watermarking in order to successfully secure models. To reduce centralized exposure and enhance data privacy, federated learning and differential privacy can also be applied. Legal frameworks, moral principles, and user education must all be put into place as part of a comprehensive security strategy. In the end, conducting empirical research and collaborating with academia and industry will be necessary to develop and validate practical defense systems.

If any of the sections are not relevant to your manuscript, please include the heading and write 'Not applicable' for that section.

Author contributions Aouatef MAHANI: Writing-original draft. Okba Kazar: Writing- review and editing.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

1. Microsoft Azure Machine Learning. Accessed June 24, (2024)
2. Google Prediction API: Developer's Guide. https://cloudacademy.com/blog/google-prediction-api, Accessed June 24, (2024)
3. Zhang, H., Li, Y., Huang, Y., Wen, Y., Yin, J., Guan, K.: Mlmodelci: An Automatic Cloud Platform for Efficient MLaaS. In: 28th ACM International Conference on Multimedia (2020)
4. Pal, S., Gupta, Y., Shukla, A., Kanade, A., Shevade, S.: A framework for the extraction of deep neural networks by leveraging public data. arXiv preprint arXiv:1905.09165 (2019)
5. Jia, H., Choquette-Choo, C.A., Chandrasekaran, V., Papernot, N.: Entangled watermarks as a defense against model extraction. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 1937–1945 (2021)
6. Tramèr, F., Zhang, F., Juels, A., Reiter, M.K., Ristenpart, T.: Stealing machine learning models via prediction APIs. In: 25th USENIX Security Symposium (USENIX Security 16), pp. 601–618 (2016)
7. Orekondy, T., Schiele, B., Fritz, M.: Prediction poisoning: Towards defenses against dnn model stealing attacks. arXiv preprint arXiv:1906.10908 (2019)
8. Pal, S., Gupta, Y., Shukla, A., Kanade, A., Shevade, S., Ganapathy, V.: Activethief: Model Extraction Using Active Learning and Unannotated Public Data. In: AAAI Conference on Artificial Intelligence, pp. 865–872 (2020)
9. Lewis, D.D.: A sequential algorithm for training text classifiers: Corrigendum and (1994)
10. Sener, O., Savarese, S.: Active learning for convolutional neural networks: A core-set approach. arXiv preprint arXiv:1708.00489 (2017)
11. Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z.B., Swami, A.: Practical black-box attacks against machine learning. In: Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security, pp. 506–519 (2017)
12. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. arXiv preprint arXiv:1412.6572 (2014)
13. Papernot, N., McDaniel, P., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A.: The limitations of deep learning in adversarial settings. In: 2016 IEEE European Symposium on Security and Privacy (EuroS &P), pp. 372–387 (2016). IEEE
14. Yu, H., Yang, K., Zhang, T., Tsai, Y.-Y., Ho, T.-Y., Jin, Y.: Cloudleak: Large-scale deep learning models stealing through adversarial examples. In: NDSS, vol. 38, p. 102 (2020)
15. Zhou, M., Wu, J., Liu, Y., Liu, S., Zhu, C.: Dast: Data-free substitute training for adversarial attacks. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 234–243 (2020)
16. Juuti, M., Szyller, S., Marchal, S., Asokan, N.: Prada: protecting against dnn model stealing attacks. In: 2009 IEEE European Symposium on Security and Privacy, pp. 512–527 (2019)
17. Kesarwani, M., Mukhoty, B., Arya, V., Mehta, S.: Model extraction warning in mlaas paradigm. In: 34th Annual Computer Security Applications Conference, pp. 371–380 (2018)
18. Zhang, Z., Chen, Y., Wagner, D.: Seat: Similarity encoder by adversarial training for detecting model extraction attack queries. In: In: Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security, pp. 37–46 (2021)
19. Quinlan, J.R.: Induction of decision trees. Mach. Learn. **1**(1), 81–106 (1986)
20. Shapiro, S.S., Wilk, M.B.: An analysis of variance test for normality (complete samples). Biometrika **52**(3–4), 591–611 (1965)
21. Yao, H., Li, Z., Weng, H., Xue, F., Ren, K., Qin, Z.: Fdinet: Protecting against dnn model extraction via feature feature distortion index. arXiv preprint arXiv:2306.11338 (Z023)
22. Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bull. **1**(6), 80–83 (1945)

23. Kariyappa, S., Qureshi, M.K.: Defending against model stealing attacks with adaptive misinformation. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2020)
24. Kariyappa, S., Prakash, A., Qureshi, M.K.: Protecting DNNs from Theft Using an Ensemble of Diverse Models. In: International Conference on Learning Representations, pp. 1–17 (2021)
25. Zheng, H., Ye, Q., Hu, H., Fang, C., Shi, J.: BDPL: A boundary differentially private layer against machine learning model extraction attacks. In: Computer Security-ESORICS 2019: 24th European Symposium on Research in Computer Security, pp. 66–83 (2019)
26. Hendrycks, D., Mazeika, M., Dietterich, T.: Deep anomaly detection with outlier exposure. arXiv preprint arXiv:1812.04606 (2018)
27. Lee, T., Edwards, B., Molloy, I., Su, D.: Defending against model stealing attacks using deceptive perturbations. arXiv preprint arXiv:1806.00054 (2018)
28. Mazeika, M., Li, B., Forsyth, D.: How to Steer Your Adversary: Targeted and efficient model stealing defenses with gradient redirection. In: International Conference on Machine Learning, pp. 15241–15254 (2022)
29. Dziedzic, A., Kaleem, M.A., Lu, Y.S., Papernot, N.: Increasing the cost of model extraction with calibrated proof of work. arXiv preprint arXiv:2201.09243 (2022)
30. Lachtar, N., Elkhail, A.A., Bacha, A., Malik, H.: A cross-stack approach towards defending against cryptojacking. IEEE Comput. Archit. Lett. **19**(2), 126–129 (2020)
31. Papernot, N., Song, S., Mironov, I., Raghunathan, A., Talwar, K., Erlingsson, O.: Scalable private learning with pate. arXiv preprint arXiv:1802.08908 (2018)
32. Szyller, S., Atli, B.G., Marchal, S., Asokan, N.: DAWN: Dynamic adversarial watermarking of neural networks. In: 29th ACM International Conference on Multimedia, pp. 4417–4425 (2021)
33. Kornblith, S., Norouzi, M., Lee, H., Hinton, G.: Similarity of neural network representations revisited. In: International Conference on Machine Learning (PMLR), pp. 3519–3529 (2019)
34. Maini, P., Yaghini, M., Papernot, N.: Dataset inference: Ownership resolution in machine learning. arXiv preprint arXiv:2104.10706 (2021)
35. Dziedzic, A., Duan, H., Kaleem, M.A., Dhawan, N., Guan, J., Cattan, Y., Boenisch, F., Papernot, N.: Dataset inference for self-supervised models. Adv. Neural Inf. Process Syst. **35**, 12058–12070 (2022)
36. Li, Y., Zhu, L., Jia, X., Jiang, Y., Xia, S.T., Cao, X.: Defending against model stealing via verifying embedded external features. In: AAAI Conference on Artificial Intelligence, pp. 1464–1472 (2022)
37. Sachs, L.: Applied Statistics: A Handbook of Technique, 1st edn. Springer, New York (2012)
38. Krizhevsky, A.: Learning multiple layers of features from tiny images. Technical Report Technical Report, University of Toronto, Ontario, (2009)
39. Coates, A., Ng, A., Lee, H.: An Analysis of Single-layer Networks in Unsupervised Feature Learning. In: the Fourteenth International Conference on Artificial Intelligence and Statistics, pp. 215–223 (2011)
40. Deng, J., Dong, W., Socher, R., Li, J.J., Li, K., Fei-Fei, L.: Imagenet: A Large-scale Hierarchical Image Database. In: IEEE Conference on Computer Vision and Pattern Recognition, pp. 248–255 (2009)
41. LeCun, L., Bottou, L., Bengio, Y., Haffner, P.: Gradient based learning applied to document recognition. In: Proceedings of the IEEE, pp. 2278–2324 (1998)
42. Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., Ha, D.: Deep learning for classical japanese literature. arXiv preprint arXiv:1812.01718 (2018)
43. Xiao, H., Rasul, K., Vollgraf, R.: Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747 (2017)
44. Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., Ng, A.Y.: Reading Digits in Natural Images with Unsupervised Feature Learning. In: NIPS Workshop on Deep Learning and Unsupervised Feature Learning, p. 4 (2011)
45. Griffin, G., Holub, A., Perona, P.: Caltech-256 object category dataset. Technical Report Technical Report 7694, Institute of Technology, California (2007)
46. Stallkamp, J., Schlipsing, M., Salmen, J., Igel, C.: The German Traffic Sign Recognition Benchmark: a Multi-class Classification Competition. In: The 2011 International Joint Conference on Neural Networks, pp. 1453–1460 (2011)
47. Friedman, F., Hastie, T., Tibshirani, R.: The Elements of Statistical Learning, 2nd edn. Springer, New York (2009)
48. Timofte, R., Zimmermann, K., Gool, L.V.: Multi-view traffic sign detection, recognition, and 3d localization. Mach. Vis. Appl. **25**, 633–647 (2014)
49. Warden, P.: Speech commands: A dataset for limited-vocabulary speech recognition. arXiv preprint arXiv:1804.03209 (2018)
50. Maas, A., Daly, R.E., Pham, P.T., Huang, D., Ng, A.Y., Potts, C.: Learning word vectors for sentiment analysis. In: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pp. 142–150 (2011)
51. UCI machine learning repository
52. Nilsback, M.E., Zisserman, A.: A Visual Vocabulary for ?ower Classification. In: 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), pp. 1447–1454 (2006)
53. Torralba, A., Fergus, R., Freeman, W.T.: 80 million tiny images: A large data set for nonparametric object and scene recognition. IEEE Trans. Pattern Anal. Mach. Intell. **30**(11), 1958–1970 (2008)
54. Le, Y., Yang, X.: Imagenet large scale visual recognition challenge. Int. J. Comput. Vis. **115**, 211–252 (2015)
55. Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al.: Imagenet large scale visual recognition challenge. Int. J. Computer Vis. **115**, 211–252 (2015)
56. Welinder, P., Branson, S., Mita, T., Wah, C., Schroff, F., Belongie, S., P?Perona: Caltech-ucsd birds 200. Technical Report Technical Report CNS-TR-2010-001, Institute of Technology, California (2010)
57. Wah, C., Branson, S., Welinder, P., Perona, P., Belongie, S.: The caltech-ucsd birds-200-2011 dataset. Technical Report Technical Report CNS-TR-2011-001, Institute of Technology, California (2002)
58. Darlow, L.N., Crowley, E.J., Antoniou, A., Storkey, A.J.: Cinic-10 is not imagenet or cifar-10. arXiv preprint arXiv:1810.03505 (2018)
59. Liu, Z., Luo, P., Wang, X., Tang, X.: Large-scale celebfaces attributes (celeba) dataset. Retrieved August 15, 11 (2018)
60. Codella, N., Rotemberg, V., Tschandl, P., Celebi, M.E., Dusza, S., Gutman, D., Helba, B., Kalloo, A., Liopyris, K., Marchetti, M., et al.: Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the international skin imaging collaboration (isic). arXiv preprint arXiv:1902.03368 (2019)
61. BihML
62. Springenberg, J.T., Dosovitskiy, A., Brox, T., Riedmiller, M.: Striving for simplicity: The all convolutional net. arXiv preprint arXiv:1412.6806 (2014)

63. Correia-Silva, J.R., Berriel, R.F., Badue, C., Souza, A.F.D., Oliveira-Santos, T.: Copycat Cnn: Stealing Knowledge by Persuading Confession with Random Non-labeled Data. In: International Joint Conference on Neural Networks (IJCNN), p. (2018)
64. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT press (2016)
65. Truong, J.B., Maini, P., Walls, R.J., Papernot, N.: Data-free Model Extraction. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4771–4780 (2021)
66. Huang, G., Liu, Z., Maaten, L.V.D., Weinberger, K.Q.: IEEE Conference on Computer Vision and Pattern Recognition. In: Densely Connected Convolutional Networks, pp. 4700–4708 (2017)
67. Reynolds, D.A.: Gaussian Mixture Models. Encyclopedia of Biometrics, Springer (2009)
68. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531 (2015)
69. Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., Fergus, R.: Intriguing properties of neural networks. arXiv preprint arXiv:1312.6199 (2013)
70. J, D., M, C., K, L., K, T.: Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018)
71. Weng, S., Zhu, R., Wu, Y., Wang, C., Li, P., Zheng, L., Liang, D., Duann, Z.: Acceleration of high-quality raman imaging via a locality enhanced transformer network. Analyst **148**(24), 6282–6291 (2023)
72. Hilbe, J.M.: Logistic regression. Int. Encyclopedia Stat. Sci. **1**, 15–32 (2011)
73. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: Ef?cient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)
74. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: Inverted Residuals and Linear Bottlenecks. In: IEEE Conference on Computer Vision and Pattern Recognition, pp. 4510–4520 (2018)
75. Blalock, D., Ortiz, J.J.G., Frankle, J., Guttag, J.: What is the state of neural network pruning? In: Proceedings of Machine Learning and Systems, pp. 129–146 (2020)
76. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. arXiv preprint arXiv:1706.06083 (2017)
77. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
78. Carlini, N., Wagner, D.: Towards evaluating the robustness of neural networks. arXiv:1608.04644 (2017)
79. Liu, Y., She, G.R., Chen, S.Y.: Magnetic resonance image diagnosis of femoral head necrosis based on resnet18 network. Computer Methods Progr. Biomed. **208**, 106254 (2021)
80. Szegedy, C., Ioffe, S., Vanhoucke, V., Alemi, A.: Inception-resnet and the impact of residual connections on learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 4278–4284 (2017)
81. Wu, S., Zhong, S., Liu, Y.: Deep residual learning for image steganalysis. Multimedia Tools Appl. **77**, 10437–10453 (2018)
82. Mikolov, T., Joulin, A., Chopra, S., Mathieu, M., Ranzato, M.A.: Learning longer memory in recurrent neural networks. arXiv preprint arXiv:1412.7753 (2014)
83. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014)
84. Berthelot, D., Carlini, N., Goodfellow, I., Papernot, N., Oliver, A., Raffel, C.A.: Mixmatch: A holistic approach to semi-supervised learning. Adv. Neural Inf. Process Syst. **32** (2019)
85. Yin, H., Molchanov, P., Alvarez, J.M., Li, Z., Mallya, A., Hoiem, D., Jha, N., Kautz, J.: Dreaming to Distill: Data-free Knowledge Transfer Via Deep Inversion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8715–8724 (2020)
86. Meena, G., Mohbey, K.K., Indian, A., Kumar, S.: Sentiment analysis from images using vgg19 based transfer learning approach. Procedia Computer Sci. **204**, 411–418 (2022)
87. Chollet, F.: Xception: Deep Learning with Depthwise Separable Convolutions. In: Computer Vision and Pattern Recognition (CVPR), 2017 IEEE Conference On, pp. 1251–1258 (2017)
88. Wang, B., Yao, Y., Shan, S., Li, H., Viswanath, B., Zheng, H., Zhao, B.Y.: Neural Cleanse: Identifying and Mitigating Backdoor Attacks in Neural Networks. In: IEEE Symposium on Security and Privacy, IEEE, pp. 707–723 (2019)
89. Zagoruyko, S., Komodakis, N.: Wide residual networks. arXiv preprint arXiv:1605.07146 (2016)
90. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., Salakhutdinov, R.: Dropout: a simple way to prevent neural networks from overfitting. J. Mach. Learn. Res. **15**(1), 1929–1958 (2014)
91. Chiang, J.: Privacy-preserving 3-layer neural network training using mere homomorphic encryption technique. arXiv preprint arXiv:2308.09531 (2023)
92. Micaelli, P., Storkey, A.J.: Zero-shot knowledge transfer via adversarial belief matching. In: Proceedings of the 33rd International Conference on Neural Information Processing Systems, pp. 9551–9561 (2019)
93. Olah, C., Mordvintsev, A., Schubert, L.: Feature visualization. Distill **2**(11), 7 (2017)
94. Quattoni, A., Torralba, A.: Recognizing Indoor Scenes. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 413–420 (2009)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Aouatef Mahani received an engineering degree in Computer Science from the University of Sciences and Technologies Houari Boumediene, Algiers, in 2007 and the Master degree in Artificial Intelligence from the same university in 2009. Since 2010, she has been a Research Assistant at the Computer Sciences Department from the same university. Her research interests include Data Mining especially classification and clustering tasks, metaheuristics and network intrusion detection systems.



Okba Kazar obtained his Engineer's diploma, Master's, and PhD degrees in Computer Science and Artificial Intelligence from the University of Constantine. He has been a Full Professor since December 2011 at the Computer Science Department of the University of Biskra, where he played a significant role in its establishment. He is also the founder and former director of the Smart Computer Science Laboratory (LINFI) at the University of Biskra (Algeria). Throughout his career, Dr. Okba Kazar has published over 420 papers in international journals and conferences. He has authored three books: "Artificial Intelligence Manual", "Big Data Security" and "Artificial Intelligence by the practice", in addition to 17 book chapters. His primary research interests include artificial intelligence, with a focus on multi-agent systems, prognostics and health management (PHM) in medical and industrial domains, enterprise resource planning (ERP), advanced information systems, robotics, web services, semantic web, big data, the Internet of Things (IoT), cloud computing, cyber-physical systems, and digital twins. He has been a visiting professor at numerous universities across Europe, particularly in France, Spain, and the United Arab Emirates. He is currently at Kalba University (Sharjah, UAE).